

Batch Decryption without Epochs and its Application to Encrypted Mempools

Dan Boneh, Evan Laufer, and Ertem Nusret Tas

Stanford University

Abstract. Suppose Alice holds a secret key sk in a public key encryption scheme. For a given set of ciphertexts, Alice wants to create a short pre-decryption key that lets anyone decrypt this exact set of ciphertexts and nothing else. This problem is called *batch decryption*. When the secret key sk is shared among a number of decryption parties the problem is called *batch threshold decryption*. This question comes up in the context of an encrypted mempool where the goal is to publish a short pre-decryption key that can be used to decrypt all ciphertexts in a block. Prior work constructed batch threshold decryption with some limitations.

In this work, we construct three new batch decryption and batch threshold decryption schemes. We first observe that a key-policy ABE (KP-ABE) scheme directly gives a batch decryption scheme. However, the best KP-ABE schemes, which happen to be lattice-based, lead to relatively long public keys and ciphertexts. We then use very different techniques to construct a new lattice-based batch decryption scheme with shorter parameters. Our construction employs a recent preimage sampler due to Waters, Wee, and Wu. Finally, for completeness, we show that a trilinear map leads to a highly efficient threshold batch decryption scheme.

Table of Contents

1	Introduction	4
1.1	An Application to Encrypted Mempools	6
1.2	The Risk of Replicated Identities	7
1.3	A Succinctness Vulnerability (SV) for the Pre-decryption Keys	7
1.4	Additional Related Work	8
2	Technical Overview	8
2.1	Warm-up: Batch Decryption from Attribute-Based Encryption	8
2.2	Our Batch Decryption System	8
2.3	Thresholdizing the Pre-decryption Key	12
3	Preliminaries	12
3.1	Batch Decryption	13
3.2	Background on Lattices	13
3.3	Shifted Multi-Preimage Trapdoor Sampler	19
4	Explainable Discrete Gaussian Sampler SampleLeft	21
5	Our Batch Decryption System	22
5.1	Correctness and Parameter Sizes	25
5.2	Security	27
5.3	Multi-bit Encryption and Adaptive Security	32
5.4	Reducing the Parameter Sizes	33
6	Batch Decryption from Attribute Based Encryption	33
6.1	Definitions	33
6.2	The Construction	34
6.3	Correctness	35
6.4	Security	35
6.5	Parameter Sizes	36
7	Batch Decryption from Trilinear Maps	37
7.1	Background on Trilinear Maps and Generic Group Model	37
7.2	The Construction	38
7.3	Correctness	39
7.4	Thresholdizing the Secret Key	39
7.5	Security	39
8	Conclusions and open problems	43
	References	44

A	Extended Preliminaries	46
A.1	Batch Threshold Decryption	46
A.2	BEAT-MEV: Batch Threshold Decryption by Bormet, Faust, Othman, Qu	46
B	Explainable SampleLeft and the Proof of Theorem 1	49
B.1	The Explainable SampleLeft Construction	50
B.2	Proof of Theorem 1	51

1 Introduction

A public key encryption scheme that supports batch decryption is a tuple of four PPT algorithms (SETUP, ENC, PREDEC, DEC) where

- $\text{SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{pk}, \text{sk})$: takes as input a maximum batch size ℓ and outputs a key pair pk, sk ;
- $\text{ENC}(\text{pk}, r, m) \rightarrow \text{ct}$: encrypts m with respect to a unique identity tag r ;
- $\text{PREDEC}(\text{sk}, \{r_i\}_{i \in B}) \rightarrow \text{sbk}$: takes as input a set B of up to ℓ identity tags and outputs a corresponding short pre-decryption key sbk ; and
- $\text{DEC}(\text{pk}, \text{sbk}, \{\text{ct}_i\}_{i \in B}) \rightarrow \{m_i\}_{i \in B}$: uses sbk to decrypt a batch of ciphertexts $\{\text{ct}_i\}_{i \in B}$. Decryption should succeed if ct_i was properly encrypted using the identity tag r_i , for all $i \in [B]$.

In its simplest form, the encryptor can sample the identity tag r at random from a sufficiently large space to ensure that r is unique, with high probability, assuming all encryptors sample r at random. The key property of batch decryption is that the pre-decryption key sbk is short — its length should be at most polylog in the batch size $|B|$, and preferably independent of the batch size. This short pre-decryption key enables the decryption of every ciphertext that was encrypted with respect to an identity tag in $\{r_i\}_{i \in B}$, and nothing else. We give a precise definition in Section 3.1. Batch decryption is useful whenever many ciphertexts need to be decrypted concurrently by a third party. This most notably comes up in the context of an encrypted mempool, as explained in Section 1.1 below.

Remark 1. One can consider an alternative syntax where PREDEC takes as input the set of full ciphertexts to decrypt, namely, $\text{PREDEC}(\text{sk}, \{\text{ct}_i\}_{i \in B}) \rightarrow \text{sbk}$. However, we prefer the syntax where PREDEC only takes a set of identity tags as input, as defined above, since it more naturally fits with how several batch decryption schemes operate.

Batch decryption was introduced by Choudhuri, Garg, Piet, and Policharla [20]. In their scheme, however, each message was encrypted with respect to an identity tag r and an epoch id e , and only a *single* pre-decryption key can be published for every epoch e . Publishing more than one key per epoch, say for two distinct sets B_1 and B_2 , would compromise security. This is called epoch-based batch decryption, and as we explain below, it makes the scheme difficult to use in the context of an encrypted mempool. The original scheme required a new setup at the beginning of every epoch, but this was later improved [42, 1, 21] to a single one-time setup at the beginning of time (but still only one pre-decryption key is allowed per epoch).

The first batch decryption scheme without epochs, called BEAT-MEV, was proposed by Bormet, Faust, Othman, and Qu [18]. BEAT-MEV uses the alternative syntax in Remark 1, where every message is encrypted with respect to an identity tag r , notably without the need for epochs. Pre-decryption keys can be published for an unlimited number of sets B of identity tags. The difficulty with BEAT-MEV is that it has a large public key that scales at least quadratically with the size of the identity space. One can shrink the public key by restricting the identity space to the maximum batch size ℓ , however, this would require encryptors to coordinate in advance to select a unique identity tag for their messages. BEAT-MEV removes the coordination requirement by splitting transactions into multiple batches of unique identities, and setting the pre-decryption key to be a vector of keys, one key per batch. This increases the size of the pre-decryption key, and more importantly, the public key remains at least quadratic in the batch size. This also introduces a potential succinctness vulnerability (SV), where the size of sbk becomes linear in $|B|$ as discussed in Section 1.3 below and in Remark 8.

Our contributions. Our main contribution is an epoch-less batch decryption scheme with a succinct public key, ciphertexts, and pre-decryption keys.

In [Section 5](#), we present our batch decryption scheme based on the Learning with Errors (LWE) assumption. At a high level, the construction follows the structure of prior epoch-based batch decryption schemes which derive a single IBE secret key for a batch of identities. In particular, our scheme can be viewed as a composition of the the lattice IBE scheme of Agrawal, Boneh, and Boyen (ABB10) [2] with the shifted multi-preimage trapdoor sampler of Waters, Wee, and Wu (WWW25) [45, §6]. As in [2], the lattices in our construction have two parts: a trapdoor for the left lattice is used to generate the pre-decryption keys, while a trapdoor for the right lattice enables simulation of pre-decryption keys for all batches excluding the adversary’s challenge identity.

In [Section 6](#), we observe that a batch decryption scheme can be built generically using key-policy attribute-based encryption (KP-ABE), which can be constructed from LWE and its variants [14,22,46]. Although both our construction in [Section 5](#) and the KP-ABE construction in [Section 6](#) achieve public key, ciphertext, and pre-decryption key sizes that are poly-log in the batch size, our scheme has shorter parameters (see [Table 1](#)).

For completeness, in [Section 7](#), we present a very efficient batch decryption system constructed using a trilinear map. We build on the aggregate IBE scheme introduced by Goyal et al. [30, §7]. The trilinear map is used to shrink the linear-sized (in batch size) ciphertext of the pairing-based scheme in [30, §7]. The result achieves optimal parameter sizes: a single group element for the pre-decryption key, and the ciphertext is the same length as the plaintext plus a single group element.

	$ \text{pk} $	$ \text{sbk} $	$ \text{ct} $	SV
KP-ABE by BGG+14 [14]	$O_\lambda(k \log^2 \ell)$	$O_\lambda(k \log^2 \ell)$	$O_\lambda(k \log^2 \ell)$	No
KP-ABE by CW23 [22]	$O_\lambda(k \log^2 \ell)$	$\text{poly}(\lambda)$	$O_\lambda(k \log^2 \ell)$	No
KP-ABE by Wee24 [46], $\alpha = 1/3$	$k^{2/3} \text{poly}(\lambda, \log \ell)$	$\text{poly}(\lambda)$	$k^{2/3} \text{poly}(\lambda, \log \ell)$	No
KP-ABE by Wee24 [46], $\alpha = 1$	$k^2 \text{poly}(\lambda, \log \ell)$	$\text{poly}(\lambda)$	$\text{poly}(\lambda, \log \ell)$	No
BEAT-MEV – SB [18, §5.3]	$O(\ell^2 \mathbb{G}_1 + \ell \mathbb{G}_2)$	$(\log \ell) \cdot \mathbb{G}_2 $	$3 \mathbb{G}_1 + \mathbb{G}_T $	Yes
BEAT-MEV – SB*	$O(\ell^2 \mathbb{G}_1 + \ell \mathbb{G}_2)$	$ \mathbb{G}_2 $	$(\log \ell) \cdot (3 \mathbb{G}_1 + \mathbb{G}_T)$	Yes
BEAT-MEV – BLMR (Sec. A.2)	$O_\lambda(1)$	$O_\lambda(k^2)$	$O_\lambda(k^3)$	No
Our BD Scheme from LWE	$O_\lambda(k \log^2 \ell)$	$O_\lambda(\log \ell)$	$O_\lambda(k \log^2 \ell)$	No
Our BD Scheme from LWE – SB	$O_\lambda(\log^3 \ell)$	$O_\lambda(\log^2 \ell)$	$O_\lambda(\log^3 \ell)$	Yes
Our BD Scheme from LWE – SB*	$O_\lambda(\log^3 \ell)$	$O_\lambda(\log \ell)$	$O_\lambda(\log^4 \ell)$	Yes
Our BD Scheme from Trilinear Map	$ \mathbb{G}'_1 + (\ell - 1) \mathbb{G}'_2 $	$ \mathbb{G}'_3 $	$ \mathbb{G}'_1 + O_\lambda(1)$	No

Table 1: Comparison of the parameter sizes of four batch decryption systems without epochs. $|\text{pk}|$ denotes the size of the public key, $|\text{sbk}|$ denotes the size of the pre-decryption key published per batch, and $|\text{ct}|$ denotes the ciphertext size. We denote the security parameter by λ , the maximum batch size by ℓ , and the logarithm of the size of the identity space by k (so that identity tags are chosen from a set of size 2^k). We denote the sizes of the $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ group elements used by BEAT-MEV [18] by $|\mathbb{G}_i|$, $i = 1, 2, T$ and the sizes of the $\mathbb{G}'_1$, $\mathbb{G}'_2$, $\mathbb{G}'_3$ and $\mathbb{G}'_T$ group elements used by the trilinear map by $|\mathbb{G}'_i|$, for $i = 1, 2, 3, T$. To simplify the expressions, we assume that $\ell = \text{poly}(\lambda)$ and $\ell \geq \lambda$. Also note that $k > \log_2 \ell$. In the table, SB stands for sub-batches and refers to the technique in [18, §5.3], and SB* refers to the alternative method explained in [Section 5.4](#). SB and SB* remove the dependency of the public keys and ciphertexts on k , but result in a $\log(\ell)$ -factor increase in the pre-decryption key and ciphertext sizes respectively. They also introduce a succinctness vulnerability for the pre-decryption keys that can cause the size of sbk to become linear in ℓ , as discussed in [Section 1.3](#). The SV column indicates if the scheme has this succinctness vulnerability. The α parameter of the KP-ABE by Wee24 [46] enables trading-off the public key and ciphertext sizes. All of the ciphertext sizes except BEAT-MEV–SB and BEAT-MEV–SB* have an additive k term that denotes the size of the selected identity.

Comparison. In [Table 1](#), we compare the parameter sizes of four batch decryption systems. We denote the logarithm of the size of the identity space by k , and compare the parameters for the same value of k across different schemes. Our basic batch decryption scheme from LWE has an exponential improvement over BEAT-MEV in terms of public key size. Asymptotically, both schemes have similar secret key and ciphertext sizes; however, BEAT-MEV has smaller pre-decryption keys and ciphertexts in concrete terms.

In [Table 1](#), SB stands for *sub-batches* and refers to the technique described above that enables BEAT-MEV to resolve the coordination problem [18, §5.3]. SB* refers to an alternative method for resolving the same problem, explained in [Section 5.4](#). SB and SB* enable our scheme to replace the k factor in front of the public key and ciphertext sizes with $\log_2 \ell < k$. However, they are not applicable to the KP-ABE schemes. Therefore, our batch decryption scheme from LWE combined with these techniques have better parameters compared to the scheme based on the BGG+14 KP-ABE [14]. Our scheme also achieves smaller public key and ciphertext sizes compared to the batch decryption schemes obtained from the KP-ABE schemes of CW23 and Wee24 [22,14], due to the latter’s dependence on k (see [Section 6.5](#)).

Note that SB and SB* result in an extra $\log_2 \ell$ -factor increase in the pre-decryption key and ciphertext sizes respectively, for both BEAT-MEV and our batch decryption scheme from LWE. They also introduce a succinctness vulnerability discussed in [Section 1.3](#) below.

Remark 2 (lattice-based BEAT-MEV). The large public key size of BEAT-MEV is due to the use of a specific pairing-based *key-homomorphic* PRF that is also puncturable (see [Appendix A.2](#) for a description of BEAT-MEV). We observe that one can achieve a smaller public key using alternative puncturable key-homomorphic PRF constructions, such as that of Boneh, Lewi, Montgomery, and Raghunathan (BLMR) [16, §5]. Using BLMR, the base BEAT-MEV scheme would have a public key of size $O_\lambda(1)$, instead of $O(\ell^2)$ as in BEAT-MEV, but a ciphertext of size $O_\lambda(k^3)$ and a pre-decryption key of size $O_\lambda(k^2)$, which are worse than BEAT-MEV (see [Appendix A.2](#)).

1.1 An Application to Encrypted Mempools

When a transaction is posted to a blockchain, it first goes into a public mempool and waits there until it is eventually included in a block that is committed on chain. While the transaction is in the mempool, it is visible to the world and this can lead to front-running attacks that cause financial loss for the transaction issuer. This issue is called MEV [24]. An encrypted mempool [37,4] is a way to protect transactions while they are in the mempool. Instead of submitting transactions in the clear, the transaction issuer encrypts the transaction with a public key $\mathbf{pk}$ and submits it to the mempool. The corresponding secret key $\mathbf{sk}$ is shared among the nodes that operate the blockchain. Once a block is committed on chain, the nodes publish decryption shares that enable anyone to decrypt and execute all the transactions in that block. This way, a transaction becomes public only after it is included in a block that is committed on chain. Recent work [10] shows how to publish decryption shares concurrently with finalizing a block, eliminating an extra round for decryption. Encrypted mempools are already in use in some blockchains such as Osmosis and Shutter [28].

Batch decryption is a way to greatly improve the efficiency of an encrypted mempool. With a standard public key encryption, for every block, a blockchain node would need to publish ℓ decryption shares to enable the decryption of all ℓ transactions in a block. In other words, for every block the node must publish $O(\lambda\ell)$ bits to enable decryption of that block. With batch decryption, or more precisely batch threshold decryption, the node would publish a short pre-decryption key whose

length is only $O(\lambda \text{ polylog}(\ell))$ or just $O(\lambda)$. The prior batch decryption schemes [21,20,1,42,18] were proposed for exactly this purpose. In epoch-based batch decryption, the block number (a.k.a block height) would be used as the epoch. To use such a scheme, the transaction issuer would need to guess the block number where its transaction would land, and encrypt using that epoch (and random identity tag). If the issuer guesses wrong, then the transaction would never be included in a block, and must be resubmitted. Epoch-free schemes eliminate this difficulty.

When describing our construction in [Section 5](#), we focus on (non-threshold) batch decryption. In [Appendix A](#), we review the formal definition of epoch-free batch threshold decryption. In our technical overview ([Section 2.3](#)) we explain how to thresholdize the secret key in our batch decryption schemes.

1.2 The Risk of Replicated Identities

A batch decryption scheme identifies a ciphertext by its identity tag r . This makes the encrypted mempool vulnerable to an attacker who replicates the identity tag of an honest user’s transaction. Specifically, suppose an adversary observes in the mempool a ciphertext ct generated by an honest user under identity tag r . The adversary can create a *dummy* ciphertext encrypted under the *same* identity r . If this dummy ciphertext is included in a block but the honest user’s ciphertext ct is not included, then the pre-decryption key generated for that block can be used to decrypt ct before it is committed on chain. This defeats the purpose of an encrypted mempool. Solutions to this problem were discussed in the earlier works [21,20,1,42,18]. For example, the transaction issuer can generate a key pair for a signature scheme, use the public key as the identity tag, and post the encrypted transaction along with a signature on the encrypted transaction. The nodes will only include a transaction in a block if it has a valid signature with respect to its identity tag. This way, the adversary cannot post a dummy transaction with the same identity tag.

Of course, to prevent malleability attacks of a similar nature, it is crucial that batch decryption schemes provide CCA2 security. To this end, standard techniques (e.g., [12,9,11]) can be employed to elevate threshold CPA security of batch decryption to full threshold CCA2 security.

If a batch decryption protocol requires the identity tags to be selected uniformly at random, a hash function can be applied to the submitted tags before the pre-decryption key is created. When the hash function is modeled as a random oracle, the uniqueness of the tags implies a uniform distribution over their values.

1.3 A Succinctness Vulnerability (SV) for the Pre-decryption Keys

A limitation of the sub-batch methods SB and SB* mentioned in [Table 1](#) is a succinctness vulnerability, denoted by the SV column in the table. Both solutions require the encrypter to select the identity tags uniformly at random from the set $\{1, \dots, \ell\}$, so that with high probability there are at most $O(\log \ell)$ ciphertexts with the same identity tags. However, an attacker can choose the identity tags adversarially and submit ℓ ciphertexts to a batch, all with the same identity tag. This causes the pre-decryption key for these ℓ tags to be as large as $O(\ell)$ in both SB and SB*. Selecting the identity tags via a random oracle does not resolve this issue, since an efficient adversary can find ℓ pre-images that map to the same identity tag with $O(\ell^2)$ queries to the random oracle. The succinctness vulnerability is further discussed in [Remark 7](#) and [Remark 8](#).

One potential mitigation is to modify the PREDEC algorithm to cap the size of the pre-decryption key by considering only the first $O(\log \ell)$ ciphertexts with the same tag and ignore all

other ciphertexts with the same tag. However, this introduces a different vulnerability where the adversary can exclude a victim ciphertext from the batch by sending $O(\log \ell)$ malicious ciphertexts with the same identity tag as the victim ciphertext.

1.4 Additional Related Work

Early work on batch decryption [34,26] proposed to use Identity Based Encryption (IBE), where a transaction issuer encrypts to an *epoch id* as the IBE identity. Blockchain nodes then release the corresponding IBE secret key once the block at that epoch id is committed on chain. These mechanisms, however, lack the ability to selectively exclude ciphertexts from the batch, as well as any security guarantees for such excluded ciphertexts. Consequently, they are not suitable for encrypted mempools.

2 Technical Overview

2.1 Warm-up: Batch Decryption from Attribute-Based Encryption

Key-policy attribute-based encryption (KP-ABE) [38,31] generalizes the public-key encryption paradigm by associating each secret key sk_f with a function $f: \mathcal{X} \rightarrow \mathcal{Y}$. Ciphertexts are labeled with a public attribute $x \in \mathcal{X}$ and can be decrypted using sk_f if and only if $f(x) = 0$.

Batch decryption can be achieved by any KP-ABE scheme instantiated with a special class of functions. These functions f_S take an identity tag as input and return 0 if the tag belongs to a designated set of identities $S = \{r_i\}_{i \in [\ell]}$ associated with the key. Hence, a ciphertext that has a public attribute r can be decrypted by the key sk_{f_S} if and only if $r \in S$, *i.e.*, if $f_S(r) = 0$. This key sk_{f_S} functions as a pre-decryption key in a batch decryption. The complete batch decryption scheme is formally presented in [Section 6](#), where its security is shown to reduce to the security of the underlying KP-ABE scheme.

For this method to give a batch-decryption scheme we need a KP-ABE where the private key sk_{f_S} is short. The KP-ABE of Boneh et al. [14], denoted BGG+14, and its derivatives due to Cini and Wee [22] and Wee [46], all have short secret keys as needed for efficient batch decryption.

2.2 Our Batch Decryption System

2.2.1 Batch Decryption as Batch Identity Based Encryption. While KP-ABE directly implies batch decryption, it does so at the expense of large parameters (*cf.* [Table 1](#)). An alternative approach to building a practical batch decryption scheme — with epochs — is to extend an efficient identity-based encryption (IBE) scheme to the batch setting [1,21]. IBE is a public-key encryption system where public keys can be arbitrary strings [40,13]. Practical IBE schemes have been realized under various cryptographic assumptions, including the bilinear Diffie-Hellman problem [13], quadratic residuosity modulo composites [23,15], and LWE [29,2]. In the IBE framework, decryption of a ciphertext associated with an identity tag r requires the recipient to obtain the corresponding secret key sk_r from a trusted generator, which derives sk_r using a master secret key msk .

Unlike traditional IBE schemes, where each identity is associated with a distinct secret key, batch decryption schemes seek to obtain a single secret key, called the pre-decryption key, capable of decrypting ciphertexts associated with multiple identity tags (thus motivating the term batch

IBE [1]). The main requirement in this setting is to ensure that both the pre-decryption key and ciphertexts remain succinct in the size of the batch. To achieve succinct pre-decryption keys, earlier batch decryption systems (with epochs) [1,21] exploit the observation that IBE implies a signature scheme [13, §6]. Specifically, they generate pre-decryption keys in the form of a signature on an accumulator value that is a commitment to a batch of identity tags [1,21]. Then, the batch decryption functionality is realized using witness encryption for two verification relations: (i) the signature verification relation induced by the underlying IBE scheme, and (ii) a membership verification relation asserting that a given identity tag belongs to the accumulator. Here, the public inputs are the tags r , while the witness comprises the accumulator value d , a signature σ over d , and membership proofs π_r for each r with respect to d .

Concrete instantiations of this paradigm by Agarwal et al. [1], Choudhuri et al. [21] and Suegami et al. [42] employ BLS signatures for signing and KZG commitments to implement the accumulator. For witness encryption, their schemes rely on public linear constraint systems in which group elements appearing in the witness are never paired with one another. Enforcing this linearity condition requires the use of a ‘one-time BLS signature’ (without random oracles) tied to a specific ‘epoch’, thereby necessitating encryption with respect to both identity tags and a pre-determined epoch number – an essential aspect for preserving security in their constructions.

In our batch decryption scheme (without epochs), we overcome the limitation of witness encryption over linear constraint systems by moving away from pairings to lattices. Our scheme can be viewed as a batch IBE scheme based on the IBE protocol by Agrawal, Boneh, and Boyen (ABB10) [2] and the shifted multi-preimage trapdoor sampler by Waters, Wee, and Wu (WWW25) [45, §3]. We next briefly look at the main components of our construction and the steps taken towards the final scheme.

2.2.2 Shifted Multi-preimage Trapdoor Sampler by Waters, Wee, and Wu. The sampler enables obtaining a trapdoor for a collection of correlated matrices $\mathbf{A}_1, \dots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times (mk+m)}$ with marginally uniform distributions, without any hints for $\mathbf{A}_i$, $i \in [\ell]$. Concretely, for every $i \in [\ell]$, let $\mathbf{A}_i := [\mathbf{A} \mid \mathbf{B} - u_i \otimes \mathbf{G}]$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{B} \in \mathbb{Z}_q^{n \times mk}$ are uniformly-random matrices, $u_i \in \{0, 1\}^k$ are distinct bit vectors, and $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ is the gadget matrix. Here, $k \geq \lceil \log \ell \rceil$. The sampler obtains a trapdoor $\mathbf{T}_\mathbf{A}$ for the matrix

$$\mathbf{A} := \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{G} \\ & \mathbf{A}_2 & & \mathbf{G} \\ & & \ddots & \vdots \\ & & & \mathbf{A}_\ell & \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{n\ell \times (\ell mk + \ell m + m)}$$

from the public trapdoor $\mathbf{T}_\mathbf{G}$ of the gadget matrix $\mathbf{G}$. Here, the matrices $\mathbf{A}$ and $\mathbf{B}$ constitute the common reference string (CRS) of the trapdoor sampler. In our batch decryption scheme, ℓ will determine the maximum batch size.

Given $\text{td} := (\mathbf{A}, \mathbf{T}_\mathbf{A})$, one can then sample a shifted preimage for any column of vectors $(\mathbf{h}_1, \dots, \mathbf{h}_\ell) \in \mathbb{Z}_q^{n\ell}$:

$$\text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell, \hat{\mathbf{c}}) \leftarrow \text{SampleMultPre}(\text{td}, \mathbf{h}_1, \dots, \mathbf{h}_\ell) \in \mathbb{Z}_q^{\ell(mk+m)+m},$$

where $\mathbf{A}_i \mathbf{v}_i + \mathbf{G} \hat{\mathbf{c}} = \mathbf{h}_i$ for all $i \in [\ell]$. Crucially for the security of our batch decryption system, the trapdoor sampler satisfies *somewhere programmability*: given a random matrix $\mathbf{A}_{i^*}$ for index i^* , it

enables generating a succinct CRS, indistinguishable from a uniformly random one, such that for the sequence of matrices $\tilde{\mathbf{A}}_1, \dots, \tilde{\mathbf{A}}_\ell$ derived from this CRS, it holds that $\tilde{\mathbf{A}}_{i^*} = \mathbf{A}_{i^*}$.

2.2.3 The IBE Scheme by Agrawal, Boneh, and Boyen. The setup algorithm of the IBE scheme generates a matrix-trapdoor tuple $(\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T}_\mathbf{A})$, random matrices $\tilde{\mathbf{A}} \in \mathbb{Z}_q^{n \times m}$ and $\tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times m}$, a random vector $\mathbf{u} \in \mathbb{Z}_q^n$, and sets the master public key to be $\mathbf{pk} := (\mathbf{A}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \mathbf{u})$ and the master secret key to be $(\mathbf{pk}, \mathbf{T}_\mathbf{A})$. Then, to generate the secret key for an identity tag r , it computes $\mathbf{sk}_r \leftarrow \text{SampleLeft}(\mathbf{A}, \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}, \mathbf{u}, \sigma)$. Here, SampleLeft outputs a discrete Gaussian sample $\mathbf{v}$ with width σ that satisfies $[\mathbf{A} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}] \cdot \mathbf{v} = \mathbf{u}$, and $H_{\text{frd}}(r)$ outputs a matrix in $\mathbb{Z}_q^{n \times n}$. For any $r \neq r'$, the hash function H_{frd} ensures that $H_{\text{frd}}(r) - H_{\text{frd}}(r')$ is a full rank matrix.

To encrypt a bit $m \in \{0, 1\}$ for identity tag r , the encryption algorithm generates a dual-Regev ciphertext $\text{ct} := (\text{ct}[1], \text{ct}[2])$ with $\mathbf{u}$ as the public key and $[\mathbf{A} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}]$ as the public parameter:

$$\begin{aligned}\text{ct}[1] &:= \mathbf{u}^\top \mathbf{s} + \epsilon_1 + m \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}[2] &:= ([\mathbf{A} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}])^\top \mathbf{s} + \epsilon_2 \in \mathbb{Z}_q^{2m},\end{aligned}$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, and ϵ_1, ϵ_2 are low-norm noise vectors.

Finally, decryption outputs $\text{Round}(\text{ct}[1] - \mathbf{sk}_r^\top \text{ct}[2])$, where $\text{Round}(x)$ outputs 0 if $x \bmod q$ is closer to 0 than to $\lfloor q/2 \rfloor$. The scheme easily extends to encrypt a multi-bit message per ciphertext.

2.2.4 Our Batch Decryption Scheme. The public key $\mathbf{pk}$ in our batch decryption scheme contains the CRS used by the shifted multi-preimage trapdoor sampler with $k = \lceil \log \ell \rceil$ and $u_i = i$ for $i \in [\ell]$, as well as the matrices $\tilde{\mathbf{A}}, \tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times (m \lceil \log \ell \rceil + m)}$ and $\mathbf{u}$ used by the IBE scheme. The public key also contains a random matrix $\mathbf{C} \in \mathbb{Z}_q^{n \times m}$. The corresponding secret key $\mathbf{sk}$ in our scheme is a trapdoor $\mathbf{T}_\mathbf{C}$ for $\mathbf{C}$.

To encrypt a bit $m \in \{0, 1\}$ with respect to an index $i \in [\ell]$ and an identity tag r , our scheme generates a dual-Regev ciphertext $\text{ct} := (\text{ct}[1], \text{ct}[2], \text{ct}[3])$ with $\mathbf{u}$ as the public key and the matrix $[\mathbf{C} \mid \mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}]$ as the public parameter (the matrix $\mathbf{A}_i$ is obtained from the CRS in the public key $\mathbf{pk}$). The resulting ciphertext is defined as:

$$\begin{aligned}\text{ct}[1] &:= \mathbf{u}^\top \mathbf{s} + \epsilon_1 + m \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}[2] &:= \mathbf{C}^\top \mathbf{s} + \epsilon_2 \in \mathbb{Z}_q^m, \\ \text{ct}[3] &:= [\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}]^\top \mathbf{s} + \epsilon_3 \in \mathbb{Z}_q^{2(m \lceil \log \ell \rceil + m)},\end{aligned} \tag{1}$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, and $\epsilon_1, \epsilon_2, \epsilon_3$ are low-norm noise vectors. The index i is selected uniformly at random from the set $[\ell]$.

Let $\text{ct}_1, \dots, \text{ct}_\ell$ be a batch of ciphertexts where for each $i \in [\ell]$, the ciphertext ct_i is encrypted with respect to the index i and identity tag r_i . The pre-decryption algorithm PREDEC first samples a vector

$$\mathbf{v} = \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) \in \mathbb{Z}_q^{\ell(m \lceil \log \ell \rceil + m) + m + \ell(m \lceil \log \ell \rceil + m)}$$

that is a preimage of the vector $(\mathbf{u}, \dots, \mathbf{u}) \in \mathbb{Z}_q^{n\ell}$ under the matrix

$$[\mathbf{A} \mid \hat{\mathbf{A}}] := \begin{bmatrix} \mathbf{A}_1 & & \mathbf{G} & \tilde{\mathbf{A}} + H_{\text{frd}}(r_1)\tilde{\mathbf{B}} & & \\ & \mathbf{A}_2 & \mathbf{G} & & \tilde{\mathbf{A}} + H_{\text{frd}}(r_2)\tilde{\mathbf{B}} & \\ & & \ddots & & & \ddots \\ & & & \mathbf{A}_\ell & \mathbf{G} & \tilde{\mathbf{A}} + H_{\text{frd}}(r_\ell)\tilde{\mathbf{B}} \end{bmatrix}.$$

Then,

$$[\mathbf{A}_i \mid \mathbf{G} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}] \begin{bmatrix} \mathbf{v}_i^1 \\ \hat{\mathbf{c}} \\ \mathbf{v}_i^2 \end{bmatrix} = \mathbf{u} \quad \text{for all } i = 1, \dots, \ell.$$

It is possible to publicly sample this preimage $\mathbf{v}$ just given $\mathbf{pk}$ by first generating a trapdoor $\mathbf{T}_{\mathbf{A}}$ for $\mathbf{A}$ as in WWW25 [45], and then calling $\text{SampleLeft}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \hat{\mathbf{A}}, (\mathbf{u}, \dots, \mathbf{u}), \sigma)$ with that trapdoor as in ABB10 [2].

Next, to enable decryption of all ℓ ciphertexts in the batch, our pre-decryption algorithm returns the pre-decryption key $\mathbf{sbk}$ defined as

$$\mathbf{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c} := \mathbf{G}\hat{\mathbf{c}}, \sigma) \quad \text{so that } \mathbf{C} \cdot \mathbf{sbk} = \mathbf{G}\hat{\mathbf{c}}.$$

The reader may notice that $(\mathbf{sbk}, \mathbf{v}_i^1, \mathbf{v}_i^2)$ is an ABB10 secret key for the identity tag r_i . Indeed,

$$[\mathbf{C} \mid \mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}] \cdot \begin{bmatrix} \mathbf{sbk} \\ \mathbf{v}_i^1 \\ \mathbf{v}_i^2 \end{bmatrix} = \mathbf{u}.$$

Note that for decryption to succeed, every decryptor must obtain the same preimage vector $\mathbf{v} := (\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$. Hence, this vector is generated using the same sequence of random bits, obtained via a hash function modeled as a random oracle, when the decryptors and key generator invoke SampleLeft .

2.2.5 Removing the Index Dependency. The scheme described so far encrypts a message with respect to both an identity tag r and an index $i \in [\ell]$. Batch decryption can only succeed if all the indices in a batch are distinct. Indeed, given multiple ciphertexts for the same index, at most one can be used in the generation of the pre-decryption key, and thus can be decrypted by that key. To remove the need for encryptors to coordinate on their choice of index, we let the encryptor generate the matrix $\mathbf{A}_i$ in (1) as a function of the identity tag r_i , thereby removing the dependence on i . Concretely, the encryptor uses $\mathbf{A}_{r_i}$ in place of $\mathbf{A}_i$ that was generated with $u_i = i$, where

$$\mathbf{A}_{r_i} := [\mathbf{A} \mid \mathbf{B} - u_i \otimes \mathbf{G}] \quad \text{where } u_i := H(r_i) \in \{0, 1\}^k.$$

Here H is a hash function that outputs a binary string in $\{0, 1\}^k$, where $k = \beta \lceil \log(\ell) \rceil$ for some $\beta > 1$. Recall that ℓ is the maximum batch size. The rest of the encryption algorithm is unchanged.

The parameter β comes with a trade-off: larger β implies larger public parameters and ciphertexts (linear in β), as we replace each instance of $k = \lceil \log \ell \rceil$ with $k = \beta \lceil \log \ell \rceil$ in the matrix dimensions. However, it also removes the need for the encryptors to de-duplicate their indices via coordination. In Section 5.4, we propose a practical solution that obviates any coordination even when $\beta = 1$.

2.2.6 Security. In the selective IBE security game, the adversary specifies a challenge identity r^* at the outset of the game. The challenger is then responsible for answering pre-decryption key queries for batches of identity tags, with the constraint that r^* is excluded from all such batches. Importantly, the challenger must perform these simulations without knowledge of the pre-decryption keys corresponding to any batch containing r^* , and without knowledge of the trapdoors for the matrices $[\mathbf{A}_{r^*} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r^*)\tilde{\mathbf{B}}]$ and $\mathbf{C}$. This is crucial for reduction to the LWE problem.

Note that the challenger can respond to pre-decryption key queries for batches (excluding r^*) without a trapdoor for $\mathbf{C}$ by sampling a random, low-norm key sk_c . However, it still cannot sample the preimages $\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ for the matrices $[\mathbf{A}_{r_i} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}]$, $r_i \neq r^*$. To enable the simulation of these preimages, we use somewhere programability of the trapdoor sampler, along with the key sampling technique originally used to simulate the IBE secret keys in ABB10 [2, §6]. More specifically, the challenger selects $\mathbf{A}_{r^*}$ at the beginning of the protocol and generates a matching CRS for the trapdoor sampler. It then sets $\tilde{\mathbf{A}} := \mathbf{A}_{r^*}\tilde{\mathbf{R}}^* - H_{\text{frd}}(r^*)\tilde{\mathbf{B}}$, where $\tilde{\mathbf{R}}^* \leftarrow \{1, -1\}^{m \times m}$. Using a trapdoor $\mathbf{T}_{\tilde{\mathbf{B}}}$ for $\tilde{\mathbf{B}}$ (but not for $\mathbf{A}_{r_i}$, $\tilde{\mathbf{A}}$ or $\mathbf{C}$), it can invoke the algorithm `SampleRight` from [2, §4] to find $\mathbf{v}_i$, $i \in [\ell]$, as long as $r_i \neq r^*$ for any $i \in [\ell]$. Finally, it programs the random oracle that fixes the random coins of `SampleLeft`; so that calling `SampleLeft` with the programmed output results in the same sequence of vectors $(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$ as the ones simulated by the challenger.

To ensure that the challenger can identify the random bits that cause `SampleLeft` to output $(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$, our scheme uses an *explainable* version of `SampleLeft`. Recall that a randomized algorithm such as `SampleLeft` takes its usual input `inp` along with a string of random bits `rb`, and deterministically computes its output using this `rb`. We construct an associated algorithm `ExplainSL(inp, v) → rb'` that takes as input `inp` along with a vector $\mathbf{v}$ from the support of `SampleLeft`. The algorithm outputs a string `rb'` such that `SampleLeft(inp; rb')` outputs $\mathbf{v}$. We refer to `rb'` as the explanation for $\mathbf{v}$. This is discussed further in Section 4.

We present our batch decryption construction, its correctness and security proof, and estimate its parameter, ciphertext and pre-decryption key sizes in Section 5.

2.3 Thresholdizing the Pre-decryption Key

To support batch *threshold* decryption, the trapdoor for the matrix $\mathbf{C}$ can be secret shared among a set of N parties using the technique from [6, §3]. The pre-decryption keys can then only be generated by a quorum of t or more parties. We note that this technique requires two rounds of interaction between the quorum members. Since an important application for batch decryption is encrypted mempools, and blockchain nodes run consensus protocols that are already interactive, one can interleave the steps of this threshold technique with the consensus protocol rounds.

When N is small, the batch decryption scheme can be thresholdized non-interactively by applying the transformation in [17, §22.3.1]. This requires each encryptor to run a t -out-of- N secret sharing on its message m and to encrypt each message share using the public key of a different party. This method increases the ciphertext, pre-decryption key, and public parameter sizes by a factor of N .

Both these thresholdizing techniques have their limitations, making our scheme most efficient in the non-threshold settings. We leave it for future work to design better threshold techniques for our system.

3 Preliminaries

Notation. We use λ as the security parameter. We call a function $f(\cdot)$ negligible (in λ), denoted by $\text{negl}(\lambda)$, if $f(\lambda) = o(1/\text{poly}(\lambda))$ for every polynomial $\text{poly}(\cdot)$. We adopt the short-hands $\mathcal{A}$ and $\mathcal{B}$ for the adversary and Ch for the challenger. We denote computationally indistinguishable games by $\approx_c$ and statistically indistinguishable games by $\approx_s$. We denote by $[n]$ the set $\{1, \dots, n\}$ and

by $(x_{a_i})_{a_i \in \{a_1, \dots, a_k\}}$ the sequence $(x_{a_1}, \dots, x_{a_k})$ such that $a_1 < \dots < a_k$. We use $x := y$ to denote assignment. The notation $a \leftarrow S$ defines a fresh uniform random variable on the set S , unless the distribution over S is specified. For a randomized algorithm $\mathcal{A}$ we denote by $y \leftarrow \mathcal{A}(x)$ the random variable obtained by running $\mathcal{A}$ on input x . We write $y := \mathcal{A}(x; r)$ when y is the result of invoking algorithm $\mathcal{A}$ on input x with random tape r .

3.1 Batch Decryption

Definition 1. A **batch decryption** (BD) scheme instantiated with a maximum batch size of $\ell \in \mathbb{N}$, a message space $\mathcal{M}$ and an identity space $\mathcal{I}$ consists of four PPT algorithms:

- $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{pk}, \text{sk})$: The setup algorithm takes as input the security parameter λ and the batch size ℓ , and outputs a public key pk and a secret key sk .
- $\text{BD.ENC}(\text{pk}, r, m) \rightarrow \text{ct}$: The encryption algorithm takes as input a public key pk , an identity tag r and a message $m \in \mathcal{M}$, and outputs a ciphertext ct .
- $\text{BD.PREDEC}(\text{sk}, (r_i)_{i \in [\ell]}) \rightarrow \text{sbk}$: The pre-decryption algorithm takes as input the secret key sk and ℓ identity tags, and outputs a pre-decryption key sbk .
- $\text{BD.DEC}(\text{pk}, \text{sbk}, (\text{ct}_i)_{i \in [\ell]}) \rightarrow (m_i)_{i \in [\ell]}$: The decrypt algorithm takes as input the pre-decryption key sbk and ℓ ciphertext, and outputs the corresponding messages.

A BD scheme must be correct, secure under a chosen plaintext attack (CPA-security), and efficient, so that the size of the pre-decryption key sbk is succinct in the batch size ℓ .

Definition 2 (Correctness of BD). For all $\lambda \in \mathbb{N}$, $\ell \in \mathbb{N}$, $(m_i)_{i \in [\ell]} \in \mathcal{M}^\ell$ and $(r_i)_{i \in [\ell]} \in \mathcal{I}^\ell$ such that $r_i \neq r_j$ for $i \neq j$, it holds that

$$\Pr \left[\text{BD.DEC}(\text{pk}, \text{sbk}, (\text{ct}_i)_{i \in [\ell]}) = (m_i)_{i \in [\ell]} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell) \\ \text{ct}_i \leftarrow \text{BD.ENC}(\text{pk}, r_i, m_i) \quad \forall i \in [\ell] \\ \text{sbk} \leftarrow \text{BD.PREDEC}(\text{sk}, (r_i)_{i \in [\ell]}) \end{array} \right] > 1 - \text{negl}(\lambda)$$

Definition 3 (Security of BD). The security game $\text{Game}_{\mathcal{A}, b}^{\text{BD}}(1^\lambda, \ell)$ is defined in [Figure 1](#) with respect to the adversary $\mathcal{A}$. A BD scheme is selectively secure in the random oracle model, if for all $\ell = \text{poly}(\lambda)$ and PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\text{Game}_{\mathcal{A}, 0}^{\text{BD}}(1^\lambda, \ell) = 1] - \Pr[\text{Game}_{\mathcal{A}, 1}^{\text{BD}}(1^\lambda, \ell) = 1] \right| < \text{negl}(\lambda)$$

A fully secure BD scheme is defined in a similar manner, with the distinction that the adversary is allowed to select the challenge identity r^* after observing the public key pk and issuing a polynomial number of pre-challenge queries.

3.2 Background on Lattices

This section is based on lemmas from [29], [2], [33] and [45]. We assume all vectors are column vectors. Given vectors $\mathbf{v}_1, \dots, \mathbf{v}_\ell \in \mathbb{Z}^m$, we define the following short-hand notation:

$$\text{col}(\mathbf{v}_1, \dots, \mathbf{v}_\ell) := \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \dots \\ \mathbf{v}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell m}$$

The Security Game $\text{Game}_{\mathcal{A},b}^{\text{BD}}(1^\lambda, \ell)$

Challenge identity: The adversary commits to the challenge identity $r^* \in \mathcal{I}$ and sends it to the challenger.

Setup: The challenger Ch takes as input the security parameter λ and the maximum batch size ℓ . It runs $(\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell)$ and sends pk to $\mathcal{A}$. The rest of the game proceeds in rounds, as follows:

Pre-challenge queries: The adversary may issue an arbitrary number of pre-challenge queries for computing the pre-decryption key:

- $\mathcal{A}$ sends a list $(r_i)_{i \in [\ell]}$ of ℓ identity tags to Ch .
- Ch computes the pre-decryption key $\text{sbk} \leftarrow \text{BD.PREDEC}(\text{sk}, (r_i)_{i \in [\ell]})$ and sends sbk to $\mathcal{A}$.

Challenge round: At any point in time, $\mathcal{A}$ may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$ sends two messages $m_0, m_1 \in \mathcal{M}$ on which, it would like to be challenged.
- If the adversary has previously asked for the pre-decryption key corresponding to a list of identities $(r_i)_{i \in [\ell]}$ such that $r^* \in (r_i)_{i \in [\ell]}$, then Ch aborts the game.
- Otherwise, Ch computes $\text{ct}_b \leftarrow \text{BD.ENC}(\text{pk}, r^*, m_b)$ and sends ct_b to $\mathcal{A}$.

Post-challenge queries: The adversary can again send an arbitrary number of key computation queries as in the pre-challenge case. If it asks for the pre-decryption key corresponding to a list of identities $(r_i)_{i \in [\ell]}$ such that $r^* \in (r_i)_{i \in [\ell]}$, then Ch aborts the game.

Random oracle queries: Upon receiving a query containing a new input x , Ch samples a value y from the output distribution of the random oracle, stores the tuple (x, y) , and returns y . It returns the same value y upon receiving the same input x in a future query.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0, 1\}$. The game then halts with the same output b' .

Fig. 1: The security game for batch decryption.

Similarly, given the matrices $\mathbf{A}_1, \dots, \mathbf{A}_\ell \in \mathbb{Z}^{n \times m}$, we define the short-hand

$$\text{diag}(\mathbf{A}_1, \dots, \mathbf{A}_\ell) := \begin{bmatrix} \mathbf{A}_1 & & \\ & \mathbf{A}_2 & \\ & & \dots \\ & & & \mathbf{A}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell n \times \ell m}$$

We use the ℓ_2 -norm for vectors and matrices unless stated otherwise, *i.e.*, $\|\mathbf{v}\| = \|\mathbf{v}\|_2 = \sqrt{\sum_i |\mathbf{v}_i|^2}$ for a vector $\mathbf{v} = \text{col}(\mathbf{v}_1, \dots, \mathbf{v}_n) \in \mathbb{Z}^n$, and $\|\mathbf{A}\| = \max_{i \in [m]} \|\mathbf{v}_i\|$ for a matrix $\mathbf{A} = [\mathbf{v}_1, \dots, \mathbf{v}_m] \in \mathbb{Z}^{n \times m}$, where $\mathbf{v}_i \in \mathbb{Z}^n$, $i \in [m]$. We denote by $\|\mathbf{A}\|_{\text{GS}}$ the ℓ_2 -norm of the Gram-Schmidt orthogonalization of the matrix $\mathbf{A}$, and by $\mathbf{I}_n \in \mathbb{Z}^{n \times n}$ the n -by- n identity matrix. Note that for any matrix $\mathbf{T} \in \mathbb{Z}_q^{m \times m'}$, it holds that $\|\mathbf{T}\|_{\text{GS}} \leq \|\mathbf{T}\| \leq \sqrt{m} \|\mathbf{T}\|_\infty$.

For a full-rank matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the n -dimensional lattice generated by $\mathbf{A}$ is given by

$$\Lambda(\mathbf{A}) := \{\mathbf{A}\mathbf{v} : \mathbf{v} \in \mathbb{Z}^n\}.$$

When $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ for some integers n, m, q , we define the following q -ary lattices:

$$\begin{aligned} \Lambda(\mathbf{A}) &:= \{\mathbf{y} \in \mathbb{Z}_q^m : \mathbf{y} = \mathbf{A}^\top \mathbf{v} \bmod q, \mathbf{v} \in \mathbb{Z}_q^n\} \\ \Lambda^\perp(\mathbf{A}) &:= \{\mathbf{v} \in \mathbb{Z}_q^m : \mathbf{A}\mathbf{v} = \mathbf{0} \bmod q, \mathbf{v} \in \mathbb{Z}_q^n\} \\ \Lambda^{\mathbf{u}}(\mathbf{A}) &:= \{\mathbf{v} \in \mathbb{Z}_q^m : \mathbf{A}\mathbf{v} = \mathbf{u} \bmod q, \mathbf{v} \in \mathbb{Z}_q^n\} \end{aligned}$$

We denote by $\lambda_1^\infty(\mathbf{A}) := \min_{\mathbf{v} \neq \mathbf{0}} \|\mathbf{A}^\top \mathbf{v}\|_\infty$, the ℓ_∞ -norm of the shortest non-zero vector in the lattice defined by the matrix $\mathbf{A}$.

Lemma 1 (Lemma 15 of [2]). *Let $\mathbf{R} \leftarrow \{-1, 1\}^{k \times m}$. Then, there exists a universal constant C such that*

$$\Pr \left[\|\mathbf{R}\| > C\sqrt{k+m} \right] < e^{-(k+m)}.$$

3.2.1 Discrete Gaussians Over Lattices. For a Gaussian width parameter $\sigma \in \mathbb{R}^+$ and dimension $n \in \mathbb{N}$, let $\rho_\sigma : \mathbb{R}^n \rightarrow \mathbb{R}$ be the Gaussian function:

$$\rho_\sigma(\mathbf{x}) := \exp \left(-\pi \|\mathbf{x}\|_2^2 / \sigma^2 \right).$$

For a coset $\mathbf{c} + \Lambda$, the Gaussian probability mass is defined as

$$\rho_\sigma(\mathbf{c} + \Lambda) := \sum_{\mathbf{x} \in \Lambda} \rho_\sigma(\mathbf{c} + \mathbf{x}).$$

We denote the discrete Gaussian distribution over $\mathbb{Z}^n$ with parameter σ by

$$\mathcal{D}_{\mathbb{Z}^n, \sigma}(\mathbf{x}) := \frac{\rho_\sigma(\mathbf{x})}{\rho_\sigma(\mathbb{Z}^n)}.$$

For a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a vector $\mathbf{v} \in \mathbb{Z}_q^n$, we define $\mathbf{A}_\sigma^{-1}(\mathbf{v})$ as the random variable $\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}_q^m, \sigma}$ conditioned on $\mathbf{A}\mathbf{x} = \mathbf{v} \bmod q$. This function extends naturally to matrices by applying $\mathbf{A}_\sigma^{-1}$ to each column individually.

Lemma 2 (From [35], Adapted from Lemma 8 of [2]). Let $q \geq 2$ and let $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ be a matrix with $m > n$. Then for $\mathbf{u} \in \mathbb{Z}_q^n$, it holds that

$$\Pr\left[\|\mathbf{x}\| > \sqrt{m}\sigma : \mathbf{x} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{u})\right] \leq \text{negl}(n).$$

Lemma 3 (Corollary 5.4 of [29]). Let n be a positive integer, q be a prime, and $m \geq 2n \log(q)$. Then, for all but a $\text{negl}(n)$ fraction of $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, and for any $\sigma \geq \omega(\sqrt{\log(m)})$, the distribution of $\mathbf{A}\mathbf{e} \bmod q$ where $\mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}_q^m, \sigma}$ is statistically close to uniform over $\mathbb{Z}_q^n$.

Lemma 4 (Discrete Gaussian Preimages, a variant of Lemma 3.6 of [45]). Let $n, m \geq n, q, t$ be parameters. For any $\ell, k = \text{poly}(n, \log(q))$, consider the matrices $\mathbf{A}_1, \dots, \mathbf{A}_\ell, \tilde{\mathbf{A}}_1, \dots, \tilde{\mathbf{A}}_\ell \in \mathbb{Z}_q^{n \times m}$, the matrices $\mathbf{B}_1, \dots, \mathbf{B}_\ell \in \mathbb{Z}_q^{n \times k}$ and a target vector $\mathbf{t} \in \mathbb{Z}_q^{n\ell}$. Here, both $\mathbf{A}_i$ and $\tilde{\mathbf{A}}_i$ are full rank, and $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$ for all $i \in [\ell]$. Define the matrix:

$$\mathbf{A} := \begin{bmatrix} \mathbf{A}_1 & & \mathbf{B}_1 \tilde{\mathbf{A}}_1 \\ & \mathbf{A}_2 & \mathbf{B}_2 \tilde{\mathbf{A}}_2 \\ & & \dots & \dots \\ & & & \mathbf{A}_\ell \mathbf{B}_\ell & \dots & \tilde{\mathbf{A}}_\ell \end{bmatrix} \quad \text{and} \quad \mathbf{t} := \begin{bmatrix} \mathbf{t}_1 \\ \mathbf{t}_2 \\ \dots \\ \mathbf{t}_\ell \end{bmatrix}$$

For all width parameters $\sigma_1 \geq (q/\sigma_0) \log(2\ell m)$, the statistical distance between the following distributions is negligible in m :

$$\{\mathbf{v} : \mathbf{v} \leftarrow \mathbf{A}_{\sigma_1}^{-1}(\mathbf{t})\} \quad \text{and} \quad \left\{ \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) : \begin{array}{l} \hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}, \sigma_1}^k \\ \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow [\mathbf{A}_i | \tilde{\mathbf{A}}_i]_{\sigma_1}^{-1}(\mathbf{t}_i - \mathbf{B}_i \hat{\mathbf{c}}) \end{array} \right\}$$

When $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$, and $\mathbf{A}_i$ and $\tilde{\mathbf{A}}_i$ are full rank for each $i \in [\ell]$, it holds that $\lambda_1^\infty(\text{diag}([\mathbf{A}_1 | \tilde{\mathbf{A}}_1], \dots, [\mathbf{A}_\ell | \tilde{\mathbf{A}}_\ell])) \geq \sigma_0$, and $\text{diag}([\mathbf{A}_1 | \tilde{\mathbf{A}}_1], \dots, [\mathbf{A}_\ell | \tilde{\mathbf{A}}_\ell])$ is full rank by [47, Lemma 2.3]. Then, Lemma 4 follows from [47, Lemma 2.10].

Lemma 5 (Lemma 5.1 of [29]). Let $m \geq 2n \log(q)$. Then, for all but at most a q^{-n} fraction of the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, the subset-sums of the columns of $\mathbf{A}$ generate $\mathbb{Z}_q^n$; i.e., for every vector $\mathbf{u} \in \mathbb{Z}_q^n$, there exists an $\mathbf{e} \in \{0, 1\}^m$ such that $\mathbf{A}\mathbf{e} = \mathbf{u} \bmod q$.

Lemma 6 (Adapted from Lemma 5.3 of [29]). Let n and prime q be positive integers, and let $m \geq 2n \log(q)$. Then, for all but at most a q^{-n} fraction of the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, it holds that $\lambda_1^\infty(\mathbf{A}) \geq \frac{q}{4}$.

3.2.2 The Gadget Matrix and Trapdoors. For $n, q \in \mathbb{N}$, the gadget matrix $\mathbf{G}_n$ is defined as follows [33]:

$$\mathbf{G}_n := \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times m'},$$

where $\mathbf{g}^\top := [1, 2, \dots, 2^{\lfloor \log(q) \rfloor}]$ and $m' := n(\lfloor \log(q) \rfloor + 1)$.

Lemma 7 (Adapted from Theorem 4.1 of [33]). For any integers $q \geq 2$, $n \geq 1$ and $m \geq n \lfloor \log(q) \rfloor$, the lattice $\Lambda^\perp(\mathbf{G})$, $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$, has a known basis $\mathbf{S} \in \mathbb{Z}^{m \times m}$ with $\|\mathbf{S}\|_{\text{GS}} \leq \sqrt{5}$ and $\|\mathbf{S}\| \leq \max\{\sqrt{5}, \sqrt{k}\}$, and preimage sampling from $\mathbf{G}_\sigma^{-1}$ with Gaussian parameter $\sigma \geq \|\mathbf{S}\|_{\text{GS}} \cdot \omega(\sqrt{\log(n)})$ can be performed in time $O(n \cdot \log^c(n))$ for some constant c .

Lemma 8 (From [29]). *Let n be a positive integer and q be a prime. Then, for any $\sigma \geq \omega(\sqrt{\log(m')})$, the distribution of $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$, where $\mathbf{v} \leftarrow \mathbb{Z}_q^n$, is statistically close to $\mathcal{D}_{\mathbb{Z}_q, \sigma}^{m'}$, where $m' = n(\lfloor \log(q) \rfloor + 1)$.*

Lemma 8 follows from the proof of [29, Corollary 5.4], along with [29, Lemma 3.1] and [33, Theorem 4.1]. Here, [33, Theorem 4.1] states that $\Lambda^\perp(\mathbf{G})$ has a known basis $\mathbf{S} \in \mathbb{Z}_q^{m' \times m'}$ such that $\|\mathbf{S}\|_{\text{GS}} \leq \sqrt{5}$. On the other hand, by [29, Lemma 3.1], the smoothing parameter η_ϵ for $\Lambda^\perp(\mathbf{G})$ is bounded by $\sqrt{5} \cdot \omega(\sqrt{\log(m')})$. This implies that for $\sigma \geq \omega(\sqrt{\log(m')})$, it holds that $\sigma \geq \eta_\epsilon(\Lambda^\perp(\mathbf{G}))$. Finally, as the columns of $\mathbf{G}$ generate $\mathbb{Z}_q^n$, by the proof of [29, Corollary 5.4], the distribution of $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$, where $\mathbf{v} \leftarrow \mathbb{Z}_q^n$, is statistically close to $\mathcal{D}_{\mathbb{Z}_q, \sigma}^{m'}$, where $m' = n(\lfloor \log(q) \rfloor + 1)$.

Lemma 9 (Adapted from [33], Theorem 3.9 of [45]). *Let n, m, q be lattice parameters and $m' = n(\lfloor \log(q) \rfloor + 1)$. Then, there exist efficient algorithms (TrapGen, SamplePre) such that;*

- **TrapGen**($1^n, q, m$) $\rightarrow (\mathbf{A}, \mathbf{T}_\mathbf{A})$: *On input the lattice dimension n , the modulus q and the number of samples m , the trapdoor-generation algorithm outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ together with a gadget trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$.*
- **SamplePre**($\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{v}, \sigma$) $\rightarrow \mathbf{u}$: *On input a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$, a target vector $\mathbf{v} \in \mathbb{Z}_q^n$ and a Gaussian width parameter σ , the preimage-sampling algorithm outputs a vector $\mathbf{u} \in \mathbb{Z}_q^m$.*

Moreover, for all $m \geq O(n \log(q))$, the above algorithms satisfy the following properties:

- **Trapdoor distribution:** *The matrix $\mathbf{A}$ output by **TrapGen**($1^n, q, m$) is statistically close to uniform. Specifically, if $(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, q, m)$ and $\mathbf{A}' \leftarrow \mathbb{Z}_q^{n \times m}$, then $\Delta(\mathbf{A}, \mathbf{A}') \leq 2^{-n}$. Moreover, $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$ and $\|\mathbf{T}_\mathbf{A}\|_\infty = 1$.*
- **Preimage sampling:** *For all matrices $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$, parameters $\sigma > 0$ and all target vectors $\mathbf{v} \in \mathbb{Z}_q^n$ in the column span of $\mathbf{A}$, the output $\mathbf{u} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{v}, \sigma)$ satisfies $\mathbf{A}\mathbf{u} = \mathbf{v}$.*
- **Preimage distribution:** *Suppose $\mathbf{T}_\mathbf{A}$ is a gadget trapdoor for $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ (i.e., $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$). Then, for all $\sigma \geq \sqrt{mm'} \|\mathbf{T}_\mathbf{A}\|_\infty \cdot \omega(\sqrt{\log(n)})$ and all target vectors $\mathbf{v} \in \mathbb{Z}_q^n$, the statistical distance between the following distributions is at most 2^{-n} : $\{\mathbf{u} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{v}, \sigma)\}$ and $\{\mathbf{u} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{v})\}$.*

For a matrix $\mathbf{V} := [\mathbf{v}_1, \dots, \mathbf{v}_\ell] \in \mathbb{Z}_q^{n \times \ell}$, we define **SamplePre**($\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{V}, \sigma$) as the algorithm that outputs the matrix where the i -th column is **SamplePre**($\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{v}_i, \sigma$).

In this work, we define trapdoors such that $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$, as opposed to the version in [33], where $\mathbf{A} \begin{bmatrix} \mathbf{T}_\mathbf{A} \\ \mathbf{I} \end{bmatrix} = \mathbf{G}$.

3.2.3 Sampling Algorithms. Let $\mathbf{A}$ and $\mathbf{B}$ denote matrices in $\mathbb{Z}_q^{n \times m}$ and $\mathbf{R}$ a matrix in $\{-1, 1\}^{m \times m}$. The BD scheme in **Section 5** requires sampling short vectors statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$ for some vector $\mathbf{u} \in \mathbb{Z}_q^n$ and matrix $\mathbf{F} := [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}] \in \mathbb{Z}_q^{n \times 2m}$. For this purpose, we use the following two algorithms from [2, §4.1] that enable sampling a preimage vector using either a short basis for $\Lambda^\perp(\mathbf{A})$ or a short basis for $\Lambda^\perp(\mathbf{B})$:

1. **SampleLeft**($\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma$) takes a short basis $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$ for $\Lambda^\perp(\mathbf{A})$, a matrix $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$ and a Gaussian width parameter σ as input, and outputs a short vector $\mathbf{v} \in \Lambda^\mathbf{u}(\mathbf{F})$ from a

distribution statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$. For this purpose, the algorithm uses Peikert's basis extension method [35] to construct a basis $\mathbf{T}_\mathbf{F}$ of the lattice $\Lambda^\perp(\mathbf{F})$ such that $\|\mathbf{T}_\mathbf{F}\|_{\text{GS}} = \|\mathbf{T}_\mathbf{A}\|_{\text{GS}}$. It then calls the algorithm $\text{SamplePre}(\mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \sigma)$ from [29] to obtain a vector statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$.

2. $\text{SampleRight}(\mathbf{A}, \mathbf{B}, \mathbf{R}, \mathbf{T}_\mathbf{B}, \mathbf{u}, \sigma)$ takes a short basis $\mathbf{T}_\mathbf{B}$ for $\Lambda^\perp(\mathbf{B})$, a uniformly-random matrix $\mathbf{R} \in \{-1, 1\}^{m \times m}$ and a Gaussian width σ , and outputs a short vector $\mathbf{v} \in \Lambda^\mathbf{u}(\mathbf{F})$ from a distribution statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}]$.

Lemma 10 (Lemma 17 of [2]). *Given $q > 2$, $m > n$ and $\sigma > \|\mathbf{T}_\mathbf{A}\|_{\text{GS}} \cdot \omega(\log(m + m_1))$, output of $\text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$ is distributed statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$.*

Lemma 11 (Lemma 19 of [2]). *Given $q > 2$, $m > n$ and $\sigma > \|\mathbf{T}_\mathbf{B}\|_{\text{GS}} \cdot s_\mathbf{R} \cdot \omega(\sqrt{\log(m)})$, output of $\text{SampleRight}(\mathbf{A}, \mathbf{B}, \mathbf{R}, \mathbf{T}_\mathbf{B}, \mathbf{u}, \sigma)$ is distributed statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $s_\mathbf{R} := \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$, and $\mathbf{F} = [\mathbf{A} \mid \mathbf{A}\mathbf{R} + \mathbf{B}]$.*

Although SampleLeft and SampleRight take as input the basis $\mathbf{T}_\mathbf{A}$ and $\mathbf{T}_\mathbf{B}$ for the lattices $\Lambda^\perp(\mathbf{B})$ and $\Lambda^\perp(\mathbf{A})$, it is possible to invoke the algorithms with gadget trapdoors $\hat{\mathbf{T}}_\mathbf{A}, \hat{\mathbf{T}}_\mathbf{B}$ such that $\mathbf{A}\hat{\mathbf{T}}_\mathbf{A} = \mathbf{G}$ and $\mathbf{B}\hat{\mathbf{T}}_\mathbf{B} = \mathbf{G}$ and obtain the corresponding basis using [19, Lemma 7.4]. Then, Lemma 10 and Lemma 11 hold when the algorithms are invoked with the gadget trapdoors as well, except that now, by the proof of [19, Lemma 7.8, eqs. 7.1 and 7.2], the lower bound on σ becomes $2m^{3/2}\|\hat{\mathbf{T}}_\mathbf{A}\|_\infty \cdot \omega(\log(m + m_1))$ in Lemma 10 and $2m^{3/2}\|\hat{\mathbf{T}}_\mathbf{B}\|_\infty \cdot s_\mathbf{R} \cdot \omega(\sqrt{\log(m)})$ in Lemma 11.

3.2.4 Leftover Hash Lemma. The following lemma follows from a generalization of the leftover hash lemma [25].

Lemma 12 (Lemma 13 of [2], from [25]). *Suppose that $m \geq (n + 1) \log(q) + \omega(\log(n))$ and $q > 2$ is prime. Let $\mathbf{R}$ be a uniformly-random matrix in $\{1, -1\}^{m \times k}$, where $k = k(n)$ is a polynomial in n . Let $\mathbf{A}$ and $\mathbf{B}$ be uniformly-random matrices in $\mathbb{Z}_q^{n \times m}$ and $\mathbb{Z}_q^{n \times k}$, respectively. Then, for all vectors $\mathbf{w} \in \mathbb{Z}_q^m$, the distribution $(\mathbf{A}, \mathbf{A}\mathbf{R}, \mathbf{R}^\top \mathbf{w})$ is statistically close to the distribution $(\mathbf{A}, \mathbf{B}, \mathbf{R}^\top \mathbf{w})$.*

Let Ψ_α denote the distribution over $\mathbb{Z}_q$ of the random variable $\lfloor qX \rfloor \bmod q$, where X is a normal random variable with mean 0 and standard deviation $\alpha/\sqrt{2\pi}$.

Lemma 13 (Lemma 12 of [2]). *Let $\mathbf{e} \in \mathbb{Z}^m$ be some vector and let $\mathbf{y} \in [0, q - 1]^m$ be an integer vector such that $\mathbf{y} \leftarrow \Psi_\alpha^m$. Then the quantity $|\mathbf{e}^\top \mathbf{y}|$, treated as an integer, satisfies the following except with probability $\text{negl}(m)$:*

$$|\mathbf{e}^\top \mathbf{y}| \leq \|\mathbf{e}\|q\alpha \cdot \omega(\log(m)) + \|\mathbf{e}\|\frac{\sqrt{m}}{2}$$

3.2.5 Learning with Errors

Definition 4 (Learning with Errors – LWE, adapted from Definition 9 of [2]). *Given a prime q , a positive integer n , and a distribution χ over $\mathbb{Z}_q$, a $(\mathbb{Z}_q, n, \chi)$ -LWE problem instance is associated with a challenge oracle $\mathcal{O}$ that is either a noisy pseudo-random sampler $\mathcal{O}_s$ for a random secret key $s \in \mathbb{Z}_q^n$, or a truly random sampler $\mathcal{O}_\$,$ behaving as follows:*

- $\mathcal{O}_s$: outputs samples of the form $(\mathbf{u}_i, \mathbf{v}_i) := (\mathbf{u}_i, \mathbf{u}_i^\top \mathbf{s} + \mathbf{x}_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{u}_i \leftarrow \mathbb{Z}_q^n$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ is a uniformly-random invariant, and $\mathbf{x}_i \in \mathbb{Z}_q$ is a fresh sample from χ .
- $\mathcal{O}_\S$: outputs truly uniform random samples from $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

An algorithm $\mathcal{A}$ decides the $(\mathbb{Z}_q, n, \chi)$ -LWE problem if for some $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, the advantage of $\mathcal{A}$, defined as

$$\text{Adv}_{\text{LWE}}[\mathcal{A}] = |\Pr[\mathcal{A}^{\mathcal{O}_s} = 1] - \Pr[\mathcal{A}^{\mathcal{O}_\S} = 1]|,$$

is non-negligible in λ .

The $(\mathbb{Z}_q, n, \chi)$ -LWE assumption states that no PPT algorithm decides the $(\mathbb{Z}_q, n, \chi)$ -LWE problem.

For some $\alpha \in (0, 1)$ and a prime q , we set the noise distribution χ to be Ψ_α , the distribution over $\mathbb{Z}_q$ of the random variable $\lfloor qX \rfloor \bmod q$, where X is a normal random variable with mean 0 and standard deviation $\alpha/\sqrt{2\pi}$. Then, as in [2], we assume the hardness of approximating the SIVP and GAPSVP problems in lattices of dimension n to within approximation factors polynomial in n . Given these assumptions and [2, Theorem 11], which in turn follows from Regev's reduction [36], we conclude that deciding the $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE problem is hard, when n/α is a polynomial in n and $q > 2\sqrt{n}/\alpha$.

3.3 Shifted Multi-Preimage Trapdoor Sampler

Our BD scheme makes use of the following shifted multi-preimage trapdoor sampler, introduced and instantiated by Waters, Wee, and Wu [45].

Definition 5 (Definitions 4.2 and 4.4 from [45]). Let ℓ be a dimension parameter, and n, t, q, σ be functions of λ and ℓ . An (n, t, q, σ) -shifted multi-preimage trapdoor sampler is a tuple of efficient algorithms with the following syntax:

- $\text{Gen}(1^\lambda, 1^\ell) \rightarrow \text{crs}$: The randomized generation algorithm outputs a common reference string crs .
- $\text{Expand}(1^\lambda, 1^\ell, \text{crs}, (u_i)_{i \in [\ell]}) \rightarrow (\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td})$: The deterministic expand algorithm takes ℓ distinct bit vectors $(u_i)_{i \in [\ell]}$, outputs matrices $\mathbf{A}_1, \dots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times t}$ and a matrix-trapdoor tuple td .
- $\text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell) \rightarrow (\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, \mathbf{c})$: Given a matrix-trapdoor tuple td and target vectors $\mathbf{t}_1, \dots, \mathbf{t}_\ell$, the randomized sampling algorithm outputs a shift $\mathbf{c} \in \mathbb{Z}_q^n$ and the preimages $\boldsymbol{\pi}_i \in \mathbb{Z}_q^t$.

The sampler must satisfy:

- **Correctness:** For all $\lambda, \ell \in \mathbb{N}$, all crs in the support of $\text{Gen}(1^\lambda, 1^\ell)$, and all $\mathbf{t}_1, \dots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$, given $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) := \text{Expand}(1^\lambda, 1^\ell, \text{crs})$, it holds that

$$\Pr[\mathbf{A}_i \boldsymbol{\pi}_i = \mathbf{t}_i + \mathbf{c} \ \forall i \in [\ell]: (\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell)] = 1.$$

- **Preimage distribution:** For all polynomials $\ell = \ell(\lambda)$, except with negligible probability over $\text{crs} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$, and $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) := \text{Expand}(1^\lambda, 1^\ell, \text{crs})$, the statistical distance between the following distributions is $\text{negl}(\lambda)$:

$$\left\{ (\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell) \right\} \text{ and } \left\{ (\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_\ell, \mathbf{c}) \left| \begin{array}{l} \mathbf{c} \leftarrow \mathbb{Z}_q^n, \\ \boldsymbol{\pi}_i \leftarrow (\mathbf{A}_i^{-1})_\sigma(\mathbf{t}_i + \mathbf{c}) \ \forall i \in [\ell] \end{array} \right. \right\}$$

- **Somewhere programmable:** There exists an efficient algorithm GenProg such that for all polynomials $\ell = \ell(\lambda)$, and ℓ distinct bit vectors $(u_i)_{i \in [\ell]}$, the following holds:

- $\text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i)$ takes a bit vector u_i and matrix $\mathbf{A}_i \in \mathbb{Z}_q^{n \times t}$, and outputs a common reference string $\tilde{\text{crs}}$.
- $\forall i \in [\ell], \forall \mathbf{A}_i \in \mathbb{Z}_q^{n \times t}$:

$$\Pr \left[\mathbf{A}_i = \tilde{\mathbf{A}}_i \mid \begin{array}{l} \tilde{\text{crs}} := \text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i), \\ (\tilde{\mathbf{A}}_1, \dots, \tilde{\mathbf{A}}_\ell, \text{td}) := \text{Expand}(1^\lambda, 1^\ell, \tilde{\text{crs}}, (u_i)_{i \in [\ell]}) \end{array} \right] = 1.$$

- The statistical distance between the following distributions is $\text{negl}(\lambda)$:

$$\{\text{crs} \leftarrow \text{Gen}(1^\lambda, 1^\ell)\} \text{ and } \left\{ \tilde{\text{crs}} \mid \begin{array}{l} \mathbf{A}_i \leftarrow \mathbb{Z}_q^{n \times t}, \\ \tilde{\text{crs}} := \text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i) \end{array} \right\}$$

- **Local expansion:** There exists an efficient deterministic algorithm ExpandLocal such that:

- $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i)$ takes a bit vector u_i and outputs $\mathbf{A}_i \in \mathbb{Z}_q^{n \times t}$.
- For all $\lambda, \ell \in \mathbb{N}$ and all crs , given

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) := \text{Expand}(1^\lambda, 1^\ell, \text{crs}, (u_i)_{i \in [\ell]}),$$

it holds that for all $i \in [\ell]$, $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i) = \mathbf{A}_i$. Moreover, $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i)$ only reads $\text{poly}(\lambda, \log(\ell))$ bits of crs and runs in time $\text{poly}(\lambda, \log(\ell))$.

3.3.1 Shifted Multi-Preimage Trapdoor Sampler Construction. We next describe the shifted multi-preimage trapdoor sampler construction by Waters, Wee, and Wu [45, Construction 4.6]. Let $n = n(\lambda, \ell)$, $q = q(\lambda, \ell)$, $\sigma = \sigma(\lambda, \ell)$, $m = 3n \lceil \log q \rceil$ and $t = m(\lceil \log \ell \rceil + 1)$. The vectors u_i are drawn from the set $\{0, 1\}^{\lceil \log \ell \rceil}$.

- $\text{Gen}(1^\lambda, 1^\ell)$: This is a randomized algorithm that samples $[\mathbf{A} \mid \mathbf{B}] \leftarrow \mathbb{Z}_q^{n \times t}$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$, and outputs $\text{crs} := [\mathbf{A} \mid \mathbf{B}]$.
- $\text{Expand}(1^\lambda, 1^\ell, \text{crs}, (u_i)_{i \in [\ell]})$: Given $\text{crs} := [\mathbf{A} \mid \mathbf{B}]$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$, the expand algorithm does the following (it is a *deterministic* algorithm):
 - For each $i \in [\ell]$, let $\mathbf{B}_i := \mathbf{B} - u_i^\top \otimes \mathbf{G}$, and

$$\mathbf{A}_i := [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] = [\mathbf{A} \mid \mathbf{B}_i] \in \mathbb{Z}_q^{n \times t}.$$

- Let $\mathbf{H} \in \{0, 1\}^{(\ell t + m) \times (\ell t + m)}$ be the permutation matrix such that:

$$\mathbf{D}_\ell := \begin{bmatrix} \mathbf{A} & \mathbf{B}_1 & & & \mathbf{G} \\ & \mathbf{A} & \mathbf{B}_2 & & \mathbf{G} \\ & & \dots & & \mathbf{G} \\ & & & \mathbf{A} & \mathbf{B}_\ell & \mathbf{G} \end{bmatrix} = \begin{bmatrix} \mathbf{A} & & & \mathbf{B}_1 & & \mathbf{G} \\ & \dots & & \dots & & \mathbf{G} \\ & & \dots & \dots & & \mathbf{G} \\ & & & \mathbf{A} & & \mathbf{B}_\ell & \mathbf{G} \end{bmatrix} \mathbf{H} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}$$

- Computes $\mathbf{T}' \leftarrow \text{StructTrapGen}(\mathbf{B}, (u_i)_{i \in [\ell]}) \in \mathbb{Z}_q^{(\ell m \lceil \log \ell \rceil + m) \times \ell m}$, where StructTrapGen is the algorithm given in [45, Lemma 4.1], and outputs a trapdoor $\mathbf{T}'$ for the following matrix such that $\|\mathbf{T}'\|_\infty = 1$:

$$\begin{bmatrix} \mathbf{B} - u_1 \otimes \mathbf{G} & & & & \mathbf{G} \\ & \mathbf{B} - u_2 \otimes \mathbf{G} & & & \mathbf{G} \\ & & \dots & & \mathbf{G} \\ & & & \mathbf{B} - u_\ell \otimes \mathbf{G} & \mathbf{G} \end{bmatrix}$$

Finally, defines the trapdoor $\mathbf{T} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$ as:

$$\mathbf{T} := \mathbf{H}^{-1} \begin{bmatrix} 0_{\ell m \times \ell m} \\ \mathbf{T}' \end{bmatrix}.$$

- The algorithm sets $\text{td} := (\mathbf{D}_\ell, \mathbf{T})$ and outputs $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td})$.
- $\text{SampleMultPre}(\text{td}, \mathbf{t}_1, \dots, \mathbf{t}_\ell)$: This is a randomized algorithm that given the trapdoor $\text{td} := (\mathbf{D}_\ell, \mathbf{T})$ and the target vectors $\mathbf{t}_1, \dots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$, defines the vector $\mathbf{t} := \text{col}(\mathbf{t}_1, \dots, \mathbf{t}_\ell)$ and outputs $(\pi_1, \dots, \pi_\ell, -\mathbf{G}\hat{\mathbf{c}})$, obtained using $\text{SamplePre} : \text{col}(\pi_1, \dots, \pi_\ell, \hat{\mathbf{c}}) \leftarrow \text{SamplePre}(\mathbf{D}_\ell, \mathbf{T}, \mathbf{t}, \sigma)$.
- $\text{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i)$: This is a deterministic algorithm that given the vector $u_i \in \{0, 1\}^{\lceil \log(\ell) \rceil}$ and the matrix $\mathbf{A}_i := [\mathbf{A} \mid \mathbf{B}]$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times m \lceil \log \ell \rceil}$, outputs $\text{crs} := [\mathbf{A} \mid \mathbf{B} + u_i^\top \otimes \mathbf{G}]$.
- $\text{ExpandLocal}(1^\lambda, \text{crs}, u_i)$: This is a deterministic algorithm that given the $\text{crs} := [\mathbf{A} \mid \mathbf{B}]$ and the vector $u_i \in \{0, 1\}^{\lceil \log(\ell) \rceil}$, outputs $\mathbf{A}_i := [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] \in \mathbb{Z}_q^{n \times t}$.

Lemma 14 (Adapted from Theorem 4.7 of [45]). *Let $n \geq \lambda$, $n = n(\lambda, \ell)$, $q = q(\lambda, \ell)$, $m = 3n \lceil \log q \rceil$, and $t = m(\lceil \log \ell \rceil + 1)$. Then for all $\sigma \geq (\ell t + m) \log(\ell n)$, Construction 4.6 [45] is an (n, t, q, σ) -shifted multi-preimage trapdoor sampler with (perfect) somewhere programmability and local expansion. The size of the CRS output by $\text{Gen}(1^\lambda, 1^\ell)$ is $nt \log(q)$.*

4 Explainable Discrete Gaussian Sampler **SampleLeft**

Our batch decryption scheme in the next section relies on the **SampleLeft** algorithm from [Section 3.2.3](#). The security proof requires this algorithm to be explainable [32]. We first define what explainable **SampleLeft** means ([Definition 6](#)) and then construct such an algorithm ([Theorem 1](#)). Recall that **SampleLeft** is a randomized algorithm that takes as input a tuple $\text{inp} := (\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$, and outputs a vector $\mathbf{v}$ such that $\mathbf{F}\mathbf{v} = \mathbf{u}$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$. Informally, we say that the algorithm is explainable if there is an efficient helper algorithm, denoted **ExplainSL**, that takes as input $(\text{inp}, \mathbf{v})$, and outputs some random bits rb such that $\text{SampleLeft}(\text{inp}; \text{rb}) = \mathbf{v}$. We refer to rb as the explanation for $\mathbf{v}$. Our definition of an explainable **SampleLeft** builds on the the definition by Champion, Hsieh and Wu [19, Def. 4.1]. The construction uses the explainable **SamplePre** algorithm from [19, Sec. 7], which in turn uses the explainable discrete Gaussian sampler by Lu and Waters [32].

Definition 6 (Explainable **SampleLeft).** *Let n, m, m_1, q denote the lattice parameters. A $(\rho, \sigma_{\text{loss}})$ -explainable **SampleLeft** sampler with randomness length $\rho(\lambda, n, m, m_1, q)$ and Gaussian width loss $\sigma_{\text{loss}}(\lambda, n, m, m_1, q)$ is a pair of efficient algorithms $\Pi_{\text{SL}} = (\text{SampleLeft}, \text{ExplainSL})$ with the following syntax:*

- $\text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb}) \rightarrow \mathbf{v}$: *Given the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$, a gadget trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$ for the matrix $\mathbf{A}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a Gaussian width parameter σ , and random bits $\text{rb} \in \{0, 1\}^{\rho(\lambda, n, m, m_1, q)}$, the sampling algorithm deterministically outputs a vector $\mathbf{v} \in \mathbb{Z}_q^{m+m_1}$.*
- $\text{ExplainSL}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma) \rightarrow \text{rb}$: *Given a precision parameter κ , the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$, a gadget trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$ for the matrix $\mathbf{A}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a preimage $\mathbf{v} \in \mathbb{Z}_q^{m+m_1}$, and a Gaussian width parameter σ , the explain algorithm outputs a bit string $\text{rb} \in \{0, 1\}^{\rho(\lambda, n, m, m_1, q)}$.*

We require the following properties to hold, except with negligible probability, for all $\lambda \in \mathbb{N}$, all matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$, $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$ such that $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$, all target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, where $\|\mathbf{u}\|_\infty \leq 2^\lambda$, and all Gaussian width parameters σ , where $\|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\text{loss}}(\lambda, n, m, m_1, q) \leq \sigma \leq 2^\lambda$:

- **Correctness:** The following distributions are statistically close:

$$\{\mathbf{v} \leftarrow \text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb}); \text{rb} \leftarrow \{0, 1\}^\rho\} \quad \text{and} \quad \{\mathbf{v} \leftarrow \mathbf{F}_\sigma^{-1}(\mathbf{v})\},$$

where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$. Moreover, for all $\mathbf{v}$ in the support of $\text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma, \text{rb})$, it holds that $\mathbf{F}\mathbf{v} = \mathbf{u}$.

- **Explainability:** For all precision parameters $\kappa \in \mathbb{N}$, the statistical distance between the following distributions is bounded by $1/\kappa + \text{negl}(\lambda)$:

- **DSampleLeft:** Sample $\text{rb} \leftarrow \{0, 1\}^\rho$ and $\mathbf{v} \leftarrow \text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb})$. Output $(\mathbf{v}, \text{rb})$.
- **DExplainLeft:** Sample $\text{rb}' \leftarrow \{0, 1\}^\rho$ and $\mathbf{v} \leftarrow \text{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb}')$. Compute

$$\text{rb} \leftarrow \text{ExplainSP}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma)$$

and output $(\mathbf{v}, \text{rb})$.

In [Appendix B](#), we build an explainable **SampleLeft** sampler called Π_{SL} using the explainable discrete Gaussian preimage sampler (*i.e.*, the explainable **SamplePre** sampler) by Champion, Hsieh and Wu [19, Section 7]. The algorithm satisfies the following theorem, whose proof is also presented in [Appendix B](#).

Theorem 1. *There exists a $(\rho, \sigma_{\text{loss}})$ -explainable **SampleLeft** sampler Π_{SL} for*

$$\rho = \text{poly}(\lambda, n, m, m_1, \log(q))$$

and

$$\sigma_{\text{loss}}(\lambda, n, m, m_1, q) = 18(m + m_1)^{3/2} \log((m + m_1)\lambda) \log(\log(q)).$$

5 Our Batch Decryption System

In this section, we present our full epochless batch decryption (BD) scheme. The scheme uses the parameter ℓ that denotes the maximum batch size, and $q, n, m, t, \beta, \sigma_1, \sigma_2$ that are specified in [Section 5.1](#). Let $\Pi_{\text{SL}} = (\text{SampleLeft}, \text{ExplainSL})$ denote the explainable **SampleLeft** sampler in [Theorem 1](#) with randomness length ρ and Gaussian width loss σ_{loss} . The batch decryption scheme makes use of three hash functions H , H_{frd} , and H_{sp} , with the following properties.

Definition 7 (The hash function H). *The hash function $H: \mathcal{I} \rightarrow \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ is an injective function that maps an identity tag $r \in \mathcal{I}$ to a bit vector in $\{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ for some $\beta \geq 1$.*

In practice, H can be a collision-resistant hash function which can be helpful if the identity tag r is a long structured string (*e.g.*, a transaction description). In this case, our scheme remains correct and secure against PPT adversaries.

Definition 8 (The hash function H_{frd} [2, Sec. 5]). *For a prime q and a positive integer $n \in \mathbb{Z}$, we say that the hash function $H_{\text{frd}}: \mathcal{I} \rightarrow \mathbb{Z}_q^{n \times n}$ has full-rank differences, or FRD, if for every $r_1, r_2 \in \mathcal{I}$, where $r_1 \neq r_2$, the matrix $H_{\text{frd}}(r_1) - H_{\text{frd}}(r_2) \in \mathbb{Z}_q^{n \times n}$ is full-rank.*

Definition 9 (The hash Function H_{sp}). *The hash function*

$$H_{\text{sp}}: \mathbb{Z}_q^{\ell n \times (\ell t + m)} \times \mathbb{Z}_q^{(\ell t + m) \times \ell m} \times \mathbb{Z}_q^{n \times t} \times \mathbb{Z}_q^{n \times t} \times \mathcal{I}^\ell \times \mathbb{Z}_q^n \times \mathbb{R} \rightarrow \mathcal{R}$$

takes as input a matrix $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}$, a gadget trapdoor $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$ for the matrix $\mathbf{A}$, matrices $\tilde{\mathbf{A}}, \tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times t}$, identity tags $(r_1, \dots, r_\ell) \in \mathcal{I}^\ell$, a vector $\mathbf{u} \in \mathbb{Z}_q^n$, and a Gaussian width $\sigma \in \mathbb{R}$. It outputs a value $r \in \mathcal{R}$.

The security proof will treat the function H_{sp} as a random oracle. We also let Π_{samp} be a (n, t, q, σ_1) -shifted multi-preimage trapdoor sampler with local expansion from [45, Constr. 4.6] (cf. Section 3.3). We denote this sampler by

$$\Pi_{\text{samp}} = (\text{Gen}, \text{GenTD}, \text{Expand}, \text{ExpandLocal}).$$

We can now describe the algorithms of our batch decryption scheme.

BD.Setup $(1^\lambda, 1^\ell)$. The setup algorithm first calculates $\text{crs}_{\text{samp}} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$. It then samples a uniformly-random vector $\mathbf{u} \leftarrow \mathbb{Z}_q^n$, two uniformly-random matrices $\tilde{\mathbf{A}} \leftarrow \mathbb{Z}_q^{n \times t}$ and $\tilde{\mathbf{B}} \leftarrow \mathbb{Z}_q^{n \times t}$, as well as a tuple $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \text{TrapGen}(1^n, q, m)$. The algorithm outputs

$$\text{pk} := (\text{crs}_{\text{samp}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \mathbf{C}, \mathbf{u}) \quad \text{sk} := (\text{pk}, \mathbf{T}_{\mathbf{C}}).$$

BD.Enc $(\text{pk}, r \in \mathcal{I}, \mathbf{m} \in \{0, 1\})$. The encryption algorithm first samples a uniformly-random vector $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ and sets $\tilde{h} := H(r) \in \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$. It then computes $\mathbf{A}_r := \text{ExpandLocal}(1^\lambda, \text{crs}_{\text{samp}}, \tilde{h}) \in \mathbb{Z}_q^{n \times t}$. It next selects the noise vectors $\epsilon_1 \leftarrow \Psi_\alpha$, $\epsilon_2 \leftarrow \Psi_\alpha^m$, and $\epsilon_{3, \text{pre}} \leftarrow \Psi_\alpha^t$, along with a matrix $\tilde{\mathbf{R}} \leftarrow \{-1, 1\}^{t \times t}$. Here, $\Psi_\alpha \in \mathbb{Z}_q$ denotes the distribution of the random variable $\lfloor qX \rfloor \bmod q$ for a normal random variable X with mean 0 and standard deviation $\alpha/\sqrt{2\pi}$. Finally, it sets $\epsilon_3 := \text{col}(\epsilon_{3, \text{pre}}, \tilde{\mathbf{R}}^\top \epsilon_{3, \text{pre}})$, $\mathbf{F} := [\mathbf{A}_r | \tilde{\mathbf{A}} + H_{\text{frd}}(r) \tilde{\mathbf{B}}] \in \mathbb{Z}_q^{n \times 2t}$, and outputs the ciphertext ct as

$$\text{ct} := (r, \text{ct}[1], \text{ct}[2], \text{ct}[3]) \quad \text{where} \quad \begin{cases} \text{ct}[1] := \mathbf{u}^\top \mathbf{s} + \epsilon_1 + \mathbf{m} \cdot \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}[2] := \mathbf{C}^\top \mathbf{s} + \epsilon_2 \in \mathbb{Z}_q^m \\ \text{ct}[3] := \mathbf{F}^\top \mathbf{s} + \epsilon_3 \in \mathbb{Z}_q^{2t} \end{cases}$$

BD.PreDec $(\text{sk}, (r_i)_{i \in [\ell]})$. The pre-decryption algorithm first sets $\tilde{h}_i := H(r_i) \in \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ for each $i \in [\ell]$. It requires that $\tilde{h}_1, \dots, \tilde{h}_\ell$ are distinct vectors; otherwise, it aborts. It then generates the matrices

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td} := (\mathbf{A}, \mathbf{T}_{\mathbf{A}})) = \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, (\tilde{h}_i)_{i \in [\ell]}),$$

using the deterministic **Expand** algorithm, where

$$\mathbf{A} := \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{G} \\ & \mathbf{A}_2 & & \mathbf{G} \\ & & \ddots & \vdots \\ & & & \mathbf{A}_\ell \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}, \quad (2)$$

and constructs $\mathbf{h} := (\mathbf{u}, \dots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$ and

$$\hat{\mathbf{A}} := \begin{bmatrix} \tilde{\mathbf{A}} + H_{\text{frd}}(r_1)\tilde{\mathbf{B}} & & & \\ & \tilde{\mathbf{A}} + H_{\text{frd}}(r_2)\tilde{\mathbf{B}} & & \\ & & \ddots & \\ & & & \tilde{\mathbf{A}} + H_{\text{frd}}(r_\ell)\tilde{\mathbf{B}} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times \ell t}. \quad (3)$$

Using these matrices, it calls the hash function H_{sp} to obtain the random coins for the **SampleLeft** algorithm,

$$\text{rb} := H_{\text{sp}}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1) \in \{0, 1\}^\rho.$$

and invokes

$$\text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) := \text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb}).$$

Finally, it samples and outputs the following vector as the pre-decryption key:

$$\text{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c} := \mathbf{G}\hat{\mathbf{c}}, \sigma_2) \in \mathbb{Z}_q^m$$

so that $\mathbf{C} \cdot \text{sbk} = \mathbf{G} \cdot \hat{\mathbf{c}}$.

BD.Dec(pk, sbk, $(\text{ct}_i)_{i \in [\ell]}$). For each $i \in [\ell]$ the decryption algorithm parses the ciphertexts as $(r_i, \text{ct}_i[1], \text{ct}_i[2], \text{ct}_i[3]) \leftarrow \text{ct}_i$. It computes $\tilde{h}_i := H(r_i) \in \{0, 1\}^{\beta \lceil \log(\ell) \rceil}$ for $i \in [\ell]$, and aborts if $\tilde{h}_1, \dots, \tilde{h}_\ell$ are not distinct vectors. Next, it generates

$$(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \mathbf{T}_{\mathbf{A}}) := \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, (\tilde{h}_i)_{i \in [\ell]})$$

and constructs the matrices $\mathbf{A}$ and $\hat{\mathbf{A}}$ as in equations (2) and (3). It then calls the hash function H_{sp} and the algorithm **SampleLeft** to obtain

$$\begin{aligned} \text{rb} &:= H_{\text{sp}}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1) \in \{0, 1\}^\rho \\ \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) &:= \text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb}). \end{aligned}$$

Finally, for each $i \in [\ell]$, it sets $\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ and outputs $\mathbf{m}'_i$ as the message:

$$\mathbf{m}'_i := \text{round}(\text{ct}_i[1] - \text{sbk}^\top \text{ct}_i[2] - \mathbf{v}_i^\top \text{ct}_i[3]).$$

Here, $\text{round}(x)$ outputs 1 if $|x - \lfloor q/2 \rfloor| < \lfloor q/4 \rfloor$, and 0 otherwise.

Remark 3 (The role of β). The parameter $\beta \in \mathbb{Z}^+$ determines the size of the vectors $\tilde{h}_i$. Since pre-decryption keys can only be generated for batches of identities $(r_i)_{i \in \ell}$ with distinct vectors $\tilde{h}_i$, when the vectors $\tilde{h}_i$ are generated using the hash function $H(\cdot)$, β must be $O(\lambda)$ to ensure that $\tilde{h}_i \neq \tilde{h}_j$ for the identities $r_i \neq r_j$ within a batch, except with negligible probability. To avoid this blow-up of parameter sizes due to β , it is possible for the encryptors to coordinate in advance, or use the sub-batches technique in [18, §5.3] (denoted by SB in Table 1). In the case of our batch decryption scheme, this technique requires using multiple sub-batches and eventually deriving pre-decryption keys for each sub-batch; so that the ciphertexts can be grouped into sub-batches with distinct vectors $\tilde{h}_i$ and decrypted (cf., [18, §5.3] for numerical analysis). For instance, setting $\beta = 1$ implies a collision of $O(\log(\ell))$ $\tilde{h}_i$ bit vectors except with negligible probability, thus necessitating the use of $O(\log(\ell))$ sub-batches, and implying a $O(\log(\ell))$ factor increase in the pre-decryption key size. In Section 5.4, we propose an alternative resolution, called SB*, where we can achieve $\beta = 1$ with no increase in the pre-decryption key size, however, at the expense of a $\log(\ell)$ -factor increase in the ciphertext size.

5.1 Correctness and Parameter Sizes

Theorem 2. *The BD scheme is correct, as in Definition 2, whenever H is an injective function as in Definition 7, and*

$$\begin{aligned} n &\geq \lambda & m &\geq 3n \lceil \log q \rceil & t &= m(\beta \lceil \log \ell \rceil + 1) & \sigma_2 &\geq m \cdot \omega(\sqrt{\log(n)}) \\ \sigma_1 &\geq \max \left(18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)), \quad O(t^2) \cdot \omega(\sqrt{\log(t)}) \right) \\ q &= O(\ell^{3/2} t^{7/2}) \cdot \omega(\log(2\ell t + m)\lambda \log(t)) \\ \alpha &= O([\ell^{3/2} t^3 \cdot \omega(\log(2\ell t + m)\lambda \log(t))]^{-1}) \end{aligned}$$

where q is rounded up to the nearest larger prime.

Proof. We first observe that as H is an injective function, when the identity tags $(r_i)_{i \in [\ell]}$ are all distinct, the bit strings $\tilde{h}_i := H(r_i)$ are also distinct. Therefore, PREDEC does not abort. Now, since $n \geq \lambda$, $q > 2$, $m \geq 3n \lceil \log q \rceil$, $t = m(\beta \lceil \log \ell \rceil + 1)$,

$$\sigma_1 \geq 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)),$$

and $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$, by Lemma 22, we make the following observations:

1. Construction 4.6 by Waters, Wee, and Wu [45] (cf. Lemma 14) is an (n, t, q, σ_1) -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion. Therefore, by the local expansion property, for all $\lambda, \ell \in \mathbb{N}$ and for all crs , given $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td}) := \text{Expand}(1^\lambda, 1^\ell, \text{crs}, (\tilde{h}_i)_{i \in [\ell]})$, it holds that $\text{ExpandLocal}(1^\lambda, \text{crs}, \tilde{h}_i) = \mathbf{A}_i$ for all $i \in [\ell]$.
2. $m \geq O(n \log(q))$ and $\sigma_2 \geq m \|\mathbf{T}_C\|_\infty \cdot \omega(\sqrt{\log(n)})$. Then, the output of the algorithms TrapGen and SamplePre in BD.SETUP and BD.PREDEC satisfy Lemma 9.
3. $\rho = \text{poly}(\lambda, n, \ell t + m, \ell t, \log(q))$ and $\sigma_1 \geq \|\mathbf{T}_A\|_\infty \cdot \sigma_{\text{loss}}$, where

$$\sigma_{\text{loss}} = 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)).$$

Therefore, by Theorem 1, $\Pi_{\text{SL}} = (\text{SampleLeft}, \text{ExplainSL})$ is correct and explainable when used with the Gaussian width parameter σ_1 .

4. For all $i \in [\ell]$ and $r \in \mathcal{I}$, the matrices $\mathbf{A}_i, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}$ and $\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}$ are full-rank (rank n), and there exists a value w such that $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}) \geq w$. Moreover, $\sigma_1 \geq (q/w) \log(2\ell t)$.

For a pre-decryption key $\mathbf{sbk}$ and ciphertext ct_i generated via the algorithms BD.PREDEC and BD.ENC, by items (1)-(2)-(3) (i.e., the properties of the shifted multi-preimage trapdoor sampler, Lemma 9 and the correctness of Π_{SL} by Theorem 1), it holds that $\mathbf{F}_{r_i} \cdot \mathbf{v}_i + \mathbf{C} \cdot \mathbf{sbk} = \mathbf{u}$, where $\mathbf{F}_{r_i} = [\mathbf{A}_i | \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}]$. Consequently, for $i \in [\ell]$,

$$\begin{aligned} &\text{ct}_i[1] - \left(\mathbf{sbk}^\top \text{ct}_i[2] + \mathbf{v}_i^\top \text{ct}_i[3] \right) \\ &= \mathbf{u}^\top \mathbf{s}_i + \epsilon_{1,i} + m_i \cdot \lfloor q/2 \rfloor \\ &\quad - \left(\mathbf{sbk}^\top \mathbf{C}^\top \mathbf{s}_i + \mathbf{sbk}^\top \epsilon_{2,i} + \mathbf{v}_i^\top \mathbf{F}_{r_i}^\top \mathbf{s}_i + \mathbf{v}_i^\top \epsilon_{3,i} \right) \\ &= (\mathbf{u} - \mathbf{C} \cdot \mathbf{sbk} - \mathbf{F}_{r_i} \cdot \mathbf{v}_i)^\top \mathbf{s}_i + \epsilon_{1,i} - \left(\mathbf{sbk}^\top \epsilon_{2,i} + \mathbf{v}_i^\top \epsilon_{3,i} \right) + m_i \cdot \lfloor q/2 \rfloor \\ &= \left(\epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} - \mathbf{v}_i^\top \epsilon_{3,i} \right) + m_i \cdot \lfloor q/2 \rfloor \end{aligned}$$

Recall that $\epsilon_{3,i} := \text{col}(\epsilon_{3,i,\text{pre}}, \tilde{\mathbf{R}}_i^\top \epsilon_{3,i,\text{pre}})$. Therefore,

$$\epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} - \mathbf{v}_i^\top \epsilon_{3,i} = \epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} - (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}$$

Next, we observe that by the correctness of Π_{SL} at item (3) (i.e., [Theorem 1](#)), output of `SampleLeft` is statistically indistinguishable from a sample from $\mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$, where $\mathbf{h} := \text{col}(\mathbf{u}, \dots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$ and $\mathbf{F} = [\mathbf{A} | \hat{\mathbf{A}}]$ (see equations (2) and (3) for $\mathbf{A}$ and $\hat{\mathbf{A}}$). Therefore, by item (4), we can invoke [Lemma 4](#) for $(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$, and infer that $\mathbf{v}_i$ is a Gaussian sample with width parameter σ_1 . Combining this with [Lemma 2](#), we observe that for all $i \in [\ell]$, $\|\mathbf{v}_i\| \leq \sqrt{2t}\sigma_1$ except with negligible probability. Then, by [Lemma 1](#), $\|\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq \|\mathbf{v}_i^1\| + \|\tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq O(t\sigma_1)$.

By item (2) (i.e., [Lemma 9](#)), we know that $\mathbf{sbk}$ is a Gaussian sample with width parameter σ_2 . Again, combining this with [Lemma 2](#), we observe that $\|\mathbf{sbk}\| \leq \sqrt{m}\sigma_2$ except with negligible probability. Hence, by [Lemma 13](#),

$$\begin{aligned} \|\epsilon_{1,i} - \mathbf{sbk}^\top \epsilon_{2,i} - (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}\| &\leq \\ &\leq |\epsilon_{1,i}| + |\mathbf{sbk}^\top \epsilon_{2,i}| + |(\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \epsilon_{3,i,\text{pre}}| \\ &\leq (t\sigma_1 + \sqrt{m}\sigma_2)q\alpha \cdot \omega(\log(t)) + O\left((t\sigma_1 + \sqrt{m}\sigma_2) \cdot \frac{\sqrt{t}}{2}\right) \end{aligned}$$

Finally, for decryption to succeed, the error term must be less than $q/5$ except with negligible probability. For this to be true, the inequality above requires that $\alpha < ((t\sigma_1 + \sqrt{m}\sigma_2) \cdot \omega(\log(t)))^{-1}$ and $q = \Omega(\sigma_1 t^{3/2} + \sigma_2 \sqrt{mt})$, which are satisfied by the values of α and q . $\square$

Given the size of the CRS, the public key scales as

$$|\mathbf{pk}| = n(4t + m) \log(q) = O(\beta \lambda^2 \log^2(\lambda) \log(\ell) + \beta \lambda^2 \log^3(\ell)),$$

the pre-decryption key scales as

$$|\mathbf{sbk}| = O(m \log(\sigma_2)) = O(\lambda \log^2(\lambda) + \lambda \log(\lambda) \log(\ell)),$$

and the ciphertexts scale as

$$|\mathbf{ct}| = (1 + m + 2t) \log(q) = O(\beta \lambda \log^2(\lambda) \log(\ell) + \beta \lambda \log^3(\ell))$$

as a function of λ , $\log(\ell)$ and β . Since $\beta \log(\ell)$ should be large enough to avoid collisions on the same vector values $\tilde{h}_i$, in [Table 1](#), $\beta \log(\ell)$ is replaced by k , which denotes the logarithm of the size of the identity space for the KP-ABE based batch decryption schemes.

Remark 4. Among the matrices $\text{crs}_{\text{samp}} = [\mathbf{A} | \mathbf{B}]$, $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$, $\mathbf{C}$ and $\mathbf{u}$ that constitute the public keys of our batch decryption system (see [Section 3.3](#) for the structure of crs_{samp}), $\mathbf{A}$, $\mathbf{B}$, $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$ and $\mathbf{u}$ are randomly-generated. Thus, they can instead be generated through a random oracle evaluation. With this modification, the public key only includes the matrix $\mathbf{C}$ of size

$$|\mathbf{pk}| = O(nm \log(q)) = O(n^2 \log^2(q)) = O(\lambda^2 \log^2(\ell) + \lambda^2 \log^2(\lambda))$$

The only change to the proof of security in [Theorem 3](#) is that whenever the simulator chooses the matrices $\mathbf{A}$, $\mathbf{B}$, $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$, and $\mathbf{u}$, it will instead program the random oracle used to generate those matrices.

5.2 Security

Theorem 3. *The BD scheme is selectively secure (Definition 3), whenever q and α are as in Theorem 2,*

$$\begin{aligned} n &\geq \lambda & m &\geq 3n \lceil \log q \rceil & \beta &= O(\lambda) & \ell &= \text{poly}(\lambda) \\ t &= m(\beta \lceil \log \ell \rceil + 1) & \sigma_2 &\geq m \cdot \omega(\sqrt{\log(n)}) & q &> 2\sqrt{n}/\alpha \\ \sigma_1 &\geq \max \left(18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)), \quad O(t^2) \cdot \omega(\sqrt{\log(t)}) \right), \end{aligned}$$

the $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE assumption holds, H_{frd} is an encoding with full-rank differences as in Definition 8, and H_{sp} is modeled as a random oracle.

Proof. Let $\mathcal{A}$ be a PPT adversary attacking BD that has advantage ϵ using polynomial number of queries to H_{st} . Let r^* denote the identity tag to be attacked. Consider the following sequence of eight hybrid games between $\mathcal{A}$ and the challenger Ch:

Hybrid 0: This is the game $\text{Game}_{\mathcal{A},b}^{\text{BD}}(1^\lambda, \ell)$ in the random oracle model:

- **Before the setup**, $\mathcal{A}$ sends a challenge identity r^* to Ch.
- **At the setup**, Ch calculates $\text{crs}_{\text{samp}} \leftarrow \text{Gen}(1^\lambda, 1^\ell)$. It then samples two uniformly-random matrices $\tilde{\mathbf{A}} \in \mathbb{Z}_q^{n \times t}$ and $\tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times t}$, a tuple $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \text{TrapGen}(1^n, q, m)$, as well as a uniformly-random vector $\mathbf{u} \in \mathbb{Z}_q^n$. It sends $\text{pk} := (\text{crs}_{\text{samp}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, \mathbf{C}, \mathbf{u})$ to $\mathcal{A}$.
- **Simulating random oracle queries to H_{sp}** : Upon receiving a new input with the matrices $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + m)}$, $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{(\ell t + m) \times \ell m}$, $\tilde{\mathbf{A}}, \tilde{\mathbf{B}} \in \mathbb{Z}_q^{n \times t}$, the identity tags $(r_i)_{i \in [\ell]} \in \mathcal{I}^\ell$, the vector $\mathbf{u} \in \mathbb{Z}_q^n$ and the Gaussian width parameter σ , Ch samples a sequence of bits $\text{rb} \leftarrow \{0, 1\}^\rho$, stores rb along with its input, and returns rb to $\mathcal{A}$.
- **To respond to pre-challenge queries** given a list of identities $(r_i)_{i \in [\ell]}$, Ch runs the algorithm $\text{BD.PREDEC}(\text{sk}, (r_i)_{i \in [\ell]})$. For this purpose, it calculates $H(r_i) = \tilde{h}_i$ for $i \in [\ell]$ and runs $(\mathbf{A}_1, \dots, \mathbf{A}_\ell, \text{td} = (\mathbf{A}, \mathbf{T}_{\mathbf{A}})) = \text{Expand}(1^\lambda, 1^\ell, \text{crs}_{\text{samp}}, (\tilde{h}_i)_{i \in [\ell]})$ to obtain the matrices $\mathbf{A}$ and $\hat{\mathbf{A}}$ as in equations (2) and (3). It then queries H_{sp} with input $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1)$ to obtain rb , invokes $\text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb})$ to obtain $\hat{\mathbf{c}}$, and returns $\text{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{G} \cdot \hat{\mathbf{c}}, \sigma_2)$.
- **Challenge**: Eventually $\mathcal{A}$ sends two messages m_0, m_1 to Ch, and receives back a ciphertext $\text{ct}^* \leftarrow \text{BD.ENC}(\text{pk}, r^*, \text{m}_b)$.
- **After some post-challenge queries**, $\mathcal{A}$ outputs a bit b' .
- The game ends with value b' .

Hybrid 1: In hybrid 1, Ch samples $\mathbf{A}_{r^*} \leftarrow \mathbb{Z}_q^{n \times t}$ and sets the CRS as $\text{crs}_{\text{samp}} \leftarrow \text{GenProg}(1^\lambda, 1^\ell, \tilde{h}^* := H(r^*), \mathbf{A}_{r^*})$.

Hybrid 2: Let $\tilde{\mathbf{R}}^* \in \{-1, 1\}^{m \times m}$ denote the random matrix sampled while creating ct^* at the challenge round. In hybrid 2, Ch chooses $\tilde{\mathbf{R}}^*$ during the setup. It then constructs $\tilde{\mathbf{A}}$ as $\tilde{\mathbf{A}} = \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(r^*) \tilde{\mathbf{B}}$.

Hybrid 3: To respond to $\mathcal{A}$'s pre- or post-challenge queries for $(r_i)_{i \in [\ell]}$, Ch first checks if $r^* \in (r_i)_{i \in [\ell]}$. If so, it aborts the game. Otherwise, it follows the same steps as in hybrid 2, except that it returns $\text{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$ as the pre-decryption key, where $\mathbf{c} = \mathbf{G} \hat{\mathbf{c}}$ and $\hat{\mathbf{c}}$ was obtained by invoking $\text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb})$.

Hybrid 4: The matrix $\mathbf{C}$ is sampled uniformly at random, $\mathbf{C} \leftarrow \mathbb{Z}_q^{n \times m}$, while the matrix $\tilde{\mathbf{B}}$ is generated along with a trapdoor $\mathbf{T}_{\tilde{\mathbf{B}}}$: $(\tilde{\mathbf{B}}, \mathbf{T}_{\tilde{\mathbf{B}}}) \leftarrow \text{TrapGen}(1^n, q, t)$.

Hybrid 5: Upon receiving a new query $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1)$ for the random oracle H_{sp} , Ch constructs the matrices $\hat{\mathbf{A}}$ and $\mathbf{A}$ as in equations (2) and (3) from the matrices crs_{samp} , $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$ and the identity tags $(r_i)_{i \in [\ell]}$. It then samples a vector $\mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$, where $\mathbf{F} = [\mathbf{A} | \hat{\mathbf{A}}]$ and $\mathbf{h} := (\mathbf{u}, \dots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$, and obtains $\text{rb} = \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1)$. It stores the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}, \text{rb})$, and returns rb to $\mathcal{A}$.

Hybrid 6: Upon receiving a new query $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1)$ for the random oracle H_{sp} , Ch does the following to respond to the query:

- Ch first checks if $r^* \in (r_i)_{i \in [\ell]}$.
- Suppose $r^* = r_{i^*}$ for some index i^* . Then, Ch samples $\mathbf{v}_{i^*} = \text{col}(\mathbf{v}_{i^*}^1, \mathbf{v}_{i^*}^2) \in \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^{2t}$ and sets $\mathbf{c} := -\mathbf{F}_{r^*} \mathbf{v}_{i^*} + \mathbf{u}$, where $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*} | \tilde{\mathbf{A}} + H_{\text{frd}}(r^*) \tilde{\mathbf{B}}]$. It then samples $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ using the algorithm in Lemma 7 and sets $\mathbf{sbk}' := \perp$.
- For $i \in [\ell]$, $i \neq i^*$, Ch obtains $\mathbf{v}_i$ as follows:

$$\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow \text{SampleRight}(\mathbf{A}_i, (H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*)) \tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{u} - \mathbf{c}, \sigma_1)$$

- Now, if $r^* \notin (r_i)_{i \in [\ell]}$, it samples $\mathbf{sbk}' \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_2}^m$ and sets $\mathbf{c} := \mathbf{C} \cdot \mathbf{sbk}'$. It then samples $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ using the algorithm in Lemma 7.
- For all $i \in [\ell]$, Ch obtains $\mathbf{v}_i$ as follows:

$$\mathbf{v}_i := \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow \text{SampleRight}(\mathbf{A}_i, (H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*)) \tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{u} - \mathbf{c}, \sigma_1)$$

- Finally, setting $\mathbf{v} := \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2)$, Ch obtains

$$\text{rb} := \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1),$$

stores the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}, \mathbf{sbk}', \text{rb})$, and returns rb to $\mathcal{A}$.

Note that Ch still responds to the pre- and post-challenge queries as in hybrid 5, instead of returning $\mathbf{sbk}'$.

Hybrid 7: Given some pre- or post-challenge query $(r_i)_{i \in [\ell]}$, Ch first checks if $r^* \in (r_i)_{i \in [\ell]}$. If so, it aborts the game. Otherwise, Ch constructs the matrix $\mathbf{A}$ as in equation (2). Without loss of generality, suppose $\mathcal{A}$ has already queried H_{sp} with the input $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1)$. In that case, Ch retrieves the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \tilde{\mathbf{A}}, \tilde{\mathbf{B}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1, \mathbf{v} := \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2), \mathbf{sbk}', \text{rb})$ and returns $\mathbf{sbk}'$.

Completing the proof. To complete the proof, let $\text{Hybrid}_i(\mathcal{A})$ denote the output distribution of hybrid i with adversary $\mathcal{A}$. We argue that the adversary's advantage changes negligibly from hybrid to hybrid, and moreover an adversary with non-negligible advantage in Hybrid 7 can be used to break the LWE assumption.

Lemma 15. *If Π_{samp} is somewhere programmable, then for all adversaries $\mathcal{A}$, $\text{Hybrid}_0(\mathcal{A}) \approx_s \text{Hybrid}_1(\mathcal{A})$.*

Proof. The only difference between these two games is the distribution of $(A_{r^*}, \text{crs}_{\text{samp}})$, which are statistically indistinguishable by somewhere programmability of Π_{samp} . $\square$

Lemma 16. *Suppose $n \geq \lambda$ and $t \geq (n + 1) \log(q) + \omega(\log(n))$. Then, for all adversaries $\mathcal{A}$, $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_2(\mathcal{A})$.*

Proof. Recall that $\tilde{\mathbf{R}}^*$ is used in the calculation of $\epsilon_{3,i}^* := \text{col}(\epsilon_{3,i,\text{pre}}^*, (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,i,\text{pre}}^*)$. Since $t \geq (n + 1) \log(q) + \omega(\log(n))$, by Lemma 12, the tuple $(\mathbf{A}_{r^*}, \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*, (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,i,\text{pre}}^*)$ has distribution statistically close to the distribution of $(\mathbf{A}_{r^*}, \mathbf{A}'_{r^*}, (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,i,\text{pre}}^*)$ for any $\epsilon_{3,i,\text{pre}}^*$, where $\mathbf{A}'_{r^*} \leftarrow \mathbb{Z}_q^{n \times t}$. Therefore, the matrix $\tilde{\mathbf{A}} := \mathbf{A}_{r^*} \tilde{\mathbf{R}}^* - H_{\text{frd}}(r^*) \tilde{\mathbf{B}}$ in hybrid 2 is statistically close to uniform, i.e., the distribution of the matrix $\tilde{\mathbf{A}}$ in hybrid 1. Thus, we can conclude that $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_2(\mathcal{A})$. $\square$

Lemma 17. *Suppose $\sigma_2 \geq m \|\mathbf{T}_{\mathbf{C}}\|_\infty \cdot \omega(\sqrt{\log(n)})$. Then, for all adversaries $\mathcal{A}$, $\text{Hybrid}_2(\mathcal{A}) \approx_s \text{Hybrid}_3(\mathcal{A})$.*

Proof. By Lemma 9, the distribution of $\text{sbk} \leftarrow \text{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c}, \sigma_2)$ in hybrid 2 is statistically indistinguishable from its distribution in hybrid 3, namely, $\text{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$. $\square$

Lemma 18. *Suppose $n \geq \lambda$ and $m, t \geq O(n \log(q))$. Then, for all PPT adversaries $\mathcal{A}$, $\text{Hybrid}_3(\mathcal{A}) \approx_s \text{Hybrid}_4(\mathcal{A})$.*

Proof. In hybrid 3, $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \text{TrapGen}(1^n, q, m)$ and $\tilde{\mathbf{B}} \leftarrow \mathbb{Z}_q^{n \times t}$, while in hybrid 4, they are sampled as $\mathbf{C} \leftarrow \mathbb{Z}_q^{n \times m}$ and $(\tilde{\mathbf{B}}, \mathbf{T}_{\tilde{\mathbf{B}}}) \leftarrow \text{TrapGen}(1^n, q, t)$. Then, by Lemma 9, the distributions of $(\tilde{\mathbf{B}}, \mathbf{C})$ are statistically indistinguishable between these two games. $\square$

Lemma 19. *Suppose $q > 2$, $t > n$, $\rho = \text{poly}(\lambda, n, \ell t + m, \ell t, \log(q))$, and $\sigma_1 \geq \|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\text{loss}}$ for $\sigma_{\text{loss}} = 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q))$. Then, for all adversaries $\mathcal{A}$, $\text{Hybrid}_4(\mathcal{A}) \approx_s \text{Hybrid}_5(\mathcal{A})$.*

Proof. By Theorem 1, $\Pi_{\text{SL}} = (\text{SampleLeft}, \text{ExplainSL})$ is correct and explainable when used with the Gaussian width parameter σ_1 . Therefore, defining $\mathbf{F} = [\mathbf{A} \mid \tilde{\mathbf{A}}]$, we observe that

$$\begin{aligned} & \left\{ (\mathbf{v}, \text{rb}') : \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h}), \text{rb}' := \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1) \right\} \\ & \approx_s \left\{ (\mathbf{v}, \text{rb}') : \text{rb} \leftarrow \{0, 1\}^\rho, \mathbf{v} := \text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb}), \text{rb}' := \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1) \right\} \\ & \approx_s \left\{ (\mathbf{v}, \text{rb}') : \text{rb}' \leftarrow \{0, 1\}^\rho, \mathbf{v} := \text{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \text{rb}') \right\}, \end{aligned}$$

where the first step follows from correctness, and the second step follows from explainability. This implies that the distribution of rb in hybrid 5 is statistically close to its distribution in hybrid 4. $\square$

Remark 5. Note that the statistical distance between the second and third distributions is bounded by $\text{negl}(\lambda) + 1/\kappa$, where $\kappa \in \mathbb{N}$ is the precision parameter. Since explainability holds for any $\kappa \in \mathbb{N}$, it can be made arbitrarily large, at the expense of a larger running time for SampleLeft .

Lemma 20. *Suppose*

1. $q > 2$, $t, m \geq 2n \log(q)$, and $\sigma_1, \sigma_2 \geq \omega(\sqrt{\log(n)})$.
2. $\sigma_1 \geq 2t^{3/2} \|\mathbf{T}_{\tilde{\mathbf{B}}}\|_\infty \cdot s_{\mathbf{R}} \cdot \omega(\sqrt{\log(t)})$, where $s_{\mathbf{R}} := \sup_{\|\mathbf{x}\|=1} \|\tilde{\mathbf{R}}^* \mathbf{x}\|$.

3. For all $i \in [\ell]$ and $r \in \mathcal{I}$, the matrices $\mathbf{A}_i$, $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$ and $\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}$ are full-rank (rank n), and there exists a value w such that $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}) \geq w$. Moreover, $\sigma_1 \geq (q/w) \log(2\ell t)$.

Then, for all adversaries $\mathcal{A}$, $\text{Hybrid}_5(\mathcal{A}) \approx_s \text{Hybrid}_6(\mathcal{A})$.

Proof. Consider the following two cases in hybrid 6:

Case 1: $r^* = r_{i^*}$ for some i^* . As $\tilde{\mathbf{A}} = \mathbf{A}_{r^*}\tilde{\mathbf{R}}^* - H_{\text{frd}}(r^*)\tilde{\mathbf{B}}$, it holds that

$$\mathbf{F}_{r^*} := [\mathbf{A}_{r^*} \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r^*)\tilde{\mathbf{B}}] = [\mathbf{A}_{r^*} \mid \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*]$$

Here, as $\mathbf{A}_{r^*}$ was drawn from a uniform distribution, $\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$ is uniform in $\mathbb{Z}_q^{n \times t}$ and independent of the other matrices by the standard leftover hash lemma. Therefore, by assumption (1) and [Lemma 3](#), $\mathbf{c} := -\mathbf{F}_{r^*} \cdot \mathbf{v}_{i^*} + \mathbf{u}$ in hybrid 6 is statistically close to a uniform sample $\mathbf{c} \leftarrow \mathbb{Z}_q^n$. Moreover, by [Lemma 8](#), for uniformly-random $\mathbf{c} \leftarrow \mathbb{Z}_q^n$, $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ is statistically close to a sample $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$.

Case 2: $r^* \notin (r_i)_{i \in [\ell]}$. Again by assumption (1) and [Lemma 3](#), $\mathbf{c} := \mathbf{C} \cdot \mathbf{s}\mathbf{b}\mathbf{k}'$ in hybrid 6 is statistically close to a uniform sample $\mathbf{c} \leftarrow \mathbb{Z}_q^n$. Again by [Lemma 8](#), $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ is statistically close to a sample $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$.

In case (2), for all $i \in [\ell]$, it holds that

$$\mathbf{F}_{r_i} := [\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}] = [\mathbf{A}_i \mid \mathbf{A}_{r^*}\tilde{\mathbf{R}}^* + (H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*))\tilde{\mathbf{B}}],$$

where $H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*) \in \mathbb{Z}_q^{n \times n}$ is full-rank (rank n) except with negligible probability, as $r_i \neq r^*$ for any $i \in [\ell]$. Thus, $(H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*))\tilde{\mathbf{B}}$ is full-rank, and $\mathbf{T}_{\tilde{\mathbf{B}}}$ acts as a trapdoor for that matrix. Then, by assumption (2) and [Lemma 11](#), the output $\mathbf{v}_i$ of

$$\text{SampleRight}(\mathbf{A}_i, (H_{\text{frd}}(r_i) - H_{\text{frd}}(r^*))\tilde{\mathbf{B}}, \tilde{\mathbf{R}}^*, \mathbf{T}_{\tilde{\mathbf{B}}}, \mathbf{u} - \mathbf{c}, \sigma_1)$$

in hybrid 6 is statistically close to a sample from $(\mathbf{F}_{r_i})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$.

By the same reasoning as above, $\mathbf{v}_i$, $i \in [\ell]$, $i \neq i^*$, are also statistically close to samples from $(\mathbf{F}_{r_i})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$ in case (1). By definition of $\mathbf{c}$ in case (1), $\mathbf{v}_{i^*}$ is statistically close to a sample from the distribution $(\mathbf{F}_{r^*})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$, i.e., $(\mathbf{F}_{r_{i^*}})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$ in case (1).

Finally, by assumption (3) and [Lemma 4](#), the statistical distance between the following distributions is negligible for $F = [\mathbf{A} \mid \hat{\mathbf{A}}]$ and $\mathbf{h} = (\mathbf{u}, \dots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$:

$$\left\{ \mathbf{v} : \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h}) \right\} \quad \text{and} \quad \left\{ \text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2) : \begin{array}{l} \hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^k \\ \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow ([\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}]_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{G}\hat{\mathbf{c}})) \end{array} \right\}$$

Hence, the distribution of $\mathbf{rb}$ in hybrid 5 is statistically close to its distribution in hybrid 6:

$$\left\{ (\mathbf{v}, \mathbf{rb}) : \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h}), \mathbf{rb} := \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1) \right\} \quad \text{and} \quad \approx_s \left\{ (\text{col}(\mathbf{v}_1^1, \dots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \dots, \mathbf{v}_\ell^2), \mathbf{rb}) : \begin{array}{l} \hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^k \\ \text{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow ([\mathbf{A}_i \mid \tilde{\mathbf{A}} + H_{\text{frd}}(r_i)\tilde{\mathbf{B}}]_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{G}\hat{\mathbf{c}})) \\ \mathbf{rb} := \text{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1) \end{array} \right\}$$

□

Lemma 21. For all PPT adversaries $\mathcal{A}$, $\text{Hybrid}_6(\mathcal{A}) \equiv \text{Hybrid}_7(\mathcal{A})$.

Proof. Since $\mathbf{sbk} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_2}^m$ and $\mathbf{c} := \mathbf{C} \cdot \mathbf{sbk}$ in hybrid 6, distribution of the pre-decryption key in hybrid 7 is indistinguishable from its distribution in hybrid 6. $\square$

Lemma 22. Suppose $n \geq \lambda$, $q > 2$, $m \geq 3n \lceil \log q \rceil$, $t = m(\beta \lceil \log \ell \rceil + 1)$,

$$\sigma_1 \geq \max \left(18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)), \quad O(t^2) \cdot \omega(\sqrt{\log(t)}) \right)$$

and $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$. Then,

1. Construction 4.6 by Waters, Wee, and Wu [45] (cf. [Lemma 14](#)) is an (n, t, q, σ_1) -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion.
2. $\rho = \text{poly}(\lambda, n, \ell t + m, \ell t, \log(q))$, and $\sigma_1 \geq \|\mathbf{T}_A\|_\infty \cdot \sigma_{\text{loss}}$ for

$$\sigma_{\text{loss}} = 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)).$$

3. $\sigma_1 \geq 2t^{3/2} \|\mathbf{T}_{\tilde{\mathbf{B}}}\|_\infty \cdot s_{\tilde{\mathbf{R}}^*} \cdot \omega(\sqrt{\log(t)}) \geq \omega(\sqrt{\log(n)})$.
4. $\sigma_2 \geq m \|\mathbf{T}_C\|_\infty \cdot \omega(\sqrt{\log(n)}) \geq \omega(\sqrt{\log(n)})$.
5. $t, m \geq O(n \log(q))$ and $t \geq (n+1) \log(q) + \omega(\log(n))$.
6. For all $i \in [\ell]$ and $r \in \mathcal{I}$, the matrices $\mathbf{A}_i$, $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$ and $\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}$ are full-rank (rank n), and there exists a value w such that $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}) \geq w$. Moreover, $\sigma_1 \geq (q/w) \log(2\ell t)$.

Here, $s_{\mathbf{R}} := \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$.

Proof. Since $n \geq \lambda$, $m = 3n \lceil \log q \rceil$, $t = m(\beta \lceil \log \ell \rceil + 1)$, and

$$\sigma_1 \geq 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q)) \geq (\ell t + m) \log(\ell n),$$

by [Lemma 14](#), construction 4.6 by Waters, Wee, and Wu [45] (cf. [Lemma 14](#)) is an (n, t, q, σ_1) -shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion (item (1)). Moreover, as $\|\mathbf{T}_A\|_\infty \leq 1$, $\sigma_1 \geq 18(2\ell t + m)^{3/2} \log((2\ell t + m)\lambda) \log(\log(q))$ satisfies $\sigma_1 \geq \|\mathbf{T}_A\|_\infty \cdot \sigma_{\text{loss}}$ (item 2).

Next, by [Lemma 9](#), $\|\mathbf{T}_{\tilde{\mathbf{B}}}\|_\infty \leq 1$. Moreover, by [Lemma 1](#) and the fact that $t \geq O(n \log(q))$, $s_{\tilde{\mathbf{R}}^*} \leq O(\sqrt{t})$. Consequently, $2t^{3/2} \|\mathbf{T}_{\tilde{\mathbf{B}}}\|_\infty \cdot s_{\tilde{\mathbf{R}}^*} \cdot \omega(\sqrt{\log(t)}) \leq O(t^2) \cdot \omega(\sqrt{\log(t)}) \leq \sigma_1$ (item 3).

Since $\|\mathbf{T}_C\|_\infty \leq 1$ by [Lemma 9](#), $\omega(\sqrt{\log(n)}) \leq m \|\mathbf{T}_C\|_\infty \cdot \omega(\sqrt{\log(n)}) \leq m \cdot \omega(\sqrt{\log(n)}) \leq \sigma_2$ (item 4).

Item 5 follows from the assumption on the sizes of m and t .

Finally, by [Lemmas 6 and 9](#), for all $i \in [\ell]$ and $r \in \mathcal{I}$, $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}} + H_{\text{frd}}(r)\tilde{\mathbf{B}}) \geq w = q/4$, which implies that $(q/w) \log(2\ell t) \leq 4 \log(2\ell t) \leq \sigma_1$. Furthermore, by [Lemmas 9 and 14](#), these matrices are full-rank (rank n). This concludes item (6). $\square$

[Lemma 22](#) implies that all previous assumptions hold, i.e., $\text{Hybrid}_1(\mathcal{A}) \approx_s \text{Hybrid}_7(\mathcal{A})$. Hence, to conclude the proof of [Theorem 3](#), we construct an adversary $\mathcal{B}$ that solves the $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE problem by acting as a challenger of hybrid 7 for $\mathcal{A}$. Upon receiving the LWE challenges $((\mathbf{y}_i, z_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q)_{i \in [t+m+1]}$, at the setup, $\mathcal{B}$ sets

$$\mathbf{u} := \mathbf{y}_0 \quad \mathbf{C} := [\mathbf{y}_1, \dots, \mathbf{y}_m] \quad \mathbf{A}_{r^*} := [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]$$

It constructs the rest of the public key as in hybrid 7, and answers the pre- and post-challenge queries as in hybrid 7.

For the challenge ciphertext, $\mathcal{B}$ sets

$$\begin{aligned} \text{ct}^*[1] &:= z_0 + \mathbf{m}_b \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \text{ct}^*[2] &:= \text{col}(z_1, \dots, z_m) \in \mathbb{Z}_q^m \\ \text{ct}^*[3] &:= \begin{bmatrix} \text{col}(z_{m+1}, \dots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^\top \text{col}(z_{m+1}, \dots, z_{m+t}) \end{bmatrix} \in \mathbb{Z}_q^{2t} \end{aligned}$$

We note that when the LWE oracle is pseudorandom (*i.e.*, $\mathcal{O} = \mathcal{O}_s$), then ct^* has the same distribution as in hybrid 7. This is because $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*} | \mathbf{A}_{r^*} \tilde{\mathbf{R}}^*]$ in hybrid 7, and by the definition of $\mathcal{O}_s$,

$$\begin{aligned} \text{ct}^*[1] &= z_0 + \mathbf{m}_b \lfloor q/2 \rfloor = (\mathbf{u})^\top \mathbf{s} + \epsilon_1^* \\ \text{ct}^*[2] &= \text{col}(z_1, \dots, z_m) = [\mathbf{y}_1, \dots, \mathbf{y}_m]^\top \mathbf{s} + \epsilon_2^* = \mathbf{C}^\top \mathbf{s} + \epsilon_2^* \\ \text{ct}^*[3] &= \begin{bmatrix} \text{col}(z_{m+1}, \dots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^\top \text{col}(z_{m+1}, \dots, z_{m+t}) \end{bmatrix} = \begin{bmatrix} [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]^\top \mathbf{s} + \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top [\mathbf{y}_{m+1}, \dots, \mathbf{y}_{m+t}]^\top \mathbf{s} + (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,\text{pre}}^* \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{r^*}^\top \mathbf{s} + \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top \mathbf{A}_{r^*}^\top \mathbf{s} + (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,\text{pre}}^* \end{bmatrix} = \mathbf{F}_{r^*}^\top \mathbf{s} + \begin{bmatrix} \epsilon_{3,\text{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top \epsilon_{3,\text{pre}}^* \end{bmatrix} = \mathbf{F}_{r^*}^\top \mathbf{s} + \epsilon_3^* \end{aligned}$$

for some short noise vectors $\epsilon_1^* \leftarrow \Psi_\alpha$, $\epsilon_2^* \leftarrow \Psi_\alpha^m$ and $\epsilon_{3,\text{pre}}^* \leftarrow \Psi_\alpha^t$.

Now, when $\mathcal{O} = \mathcal{O}_\S$, $\text{ct}^*[1]$ and $\text{ct}^*[2]$ are uniformly-random samples from $\mathbb{Z}_q$ and $\mathbb{Z}_q^m$ respectively. Similarly, $\text{col}(z_{m+1}, \dots, z_{m+t})$ is a random sample from $\mathbb{Z}_q^t$, and $(\tilde{\mathbf{R}}^*)^\top \text{col}(z_{m+1}, \dots, z_{m+t})$ is uniform in $\mathbb{Z}_q^t$ and independent of the other vectors by the standard leftover hash lemma. Here, the hash function is defined by the matrix $[\mathbf{A}_{r^*}^\top | \text{col}(z_{m+1}, \dots, z_{m+t})]$ and ensures that $\mathbf{A}_{r^*} \tilde{\mathbf{R}}^*$ and $(\tilde{\mathbf{R}}^*)^\top \text{col}(z_{m+1}, \dots, z_{m+t})$ are uniformly distributed and independent quantities. Hence, when $\mathcal{O} = \mathcal{O}_\S$, the challenge ciphertext is always uniform in $\mathbb{Z}_q \times \mathbb{Z}_q^m \times \mathbb{Z}_q^{2t}$.

Finally, we observe that if $\mathcal{A}$ has advantage ϵ , then $\mathcal{B}$ has advantage at least ϵ in deciding the $(\mathbb{Z}_q, n, \Psi_\alpha)$ -LWE problem. This concludes the proof. $\square$

5.3 Multi-bit Encryption and Adaptive Security

To encrypt messages with N bits, the public parameters can be made to include N vectors $\mathbf{u}_1, \dots, \mathbf{u}_N$, each used to encrypt one of the message bits with the same ephemeral $\mathbf{s} \in \mathbb{Z}_q^n$. This increases the size of $\text{ct}_r[1]$ by a factor of N , while keeping $\text{ct}_r[2]$ and $\text{ct}_r[3]$ the same. The security proof remains largely unchanged, except that the challenger must now query the LWE oracle $N + m + t$ times instead of $1 + m + t$ times. An alternative method would be representing the messages as integers in the range $[0, 2^\ell]$, and calculating $\text{ct}_r[1]$ as $\text{ct}_r[1] := \mathbf{u}^\top \mathbf{s}_r + \epsilon_{1,r} + \mathbf{m} \cdot \lfloor q/2^N \rfloor$. This in turn requires the error term to be less than $q/(1 + 2^{N+1})$, which increases q by a factor of 2^N . Whereas the first solution increases the ciphertext size by $O(N(\log(\ell) + \log(\lambda)))$ bits, the latter results in a factor of N increase.

To move from static to adaptive security, we can design our batch decryption system starting from the adaptively-secure version of the IBE scheme by Agrawal, Boneh, Boyen [2], which in turn is based on a similar technique by Waters [44].

5.4 Reducing the Parameter Sizes

In this section, we propose an alternative method to the sub-batches technique in [18, §5.3] to remove the coordination requirement while reducing β to 1. This technique, denoted by SB^* , does not lead to any (non-constant) increase in the pre-decryption key size, but comes at the expense of a $\log(\ell)$ -factor increase in the ciphertext size. Just like the technique in [18, §5.3], it is applicable to both our batch decryption system and BEAT-MEV.

Suppose each encryptor creates $2\lceil\log(\ell)\rceil$ ciphertexts for its message using $2\lceil\log(\ell)\rceil$ distinct identity tags and bit vectors. Then, upon receiving ciphertexts from $\ell/2$ encryptors, the pre-decryption algorithm tries to assign a unique bit vector $\tilde{h}_i$ to each encryptor from among the $2\lceil\log(\ell)\rceil$ bit vectors corresponding to the encryptor's $2\lceil\log(\ell)\rceil$ ciphertexts. This is a classical problem of balls and bins, and when all of the $\ell/2$ encryptors propose $2\lceil\log(\ell)\rceil$ choices, each can be assigned to a unique index with high probability, using an online algorithm (*cf.* [3, Thm. 4]). Therefore, the pre-decryption algorithm succeeds in its assignment with high probability, and pads the remaining $\ell/2$ positions in the batch with ciphertexts encrypted under random identity tags and the remaining $\ell/2$ distinct bit vectors. This solution results in a $2\log(\ell)$ factor increase in the size of ciphertexts, as well as an effective $2\times$ increase in the size of pre-decryption keys (given $\beta = 1$).

Remark 6 (Shrinking the ciphertext). When our batch decryption scheme is used for an encrypted mempool, it is not necessary for all of the $2\lceil\log \ell\rceil$ ciphertexts encrypting the same message to be posted on chain. Only the block proposer needs to receive these ciphertexts to select a unique bit vector $\tilde{h}_i$ for each message (transaction). After one of the $2\lceil\log \ell\rceil$ ciphertexts is placed in the block for each message, the rest of the ciphertexts can be discarded. To create the pre-decryption key and to decrypt the ciphertexts within a block, the other blockchain nodes do not need any more data than the pre-decryption key. This removes the $2\log(\ell)$ -factor increase in the size of ciphertexts for all blockchain nodes except the block proposer. Since this is a blockchain-specific observation, in Table 1, we do not consider this effective reduction in the ciphertext size for the other blockchain nodes.

Remark 7 (Succinctness Vulnerability for the Pre-decryption Key Size in SB^).* Note that when $\beta = 1$, an adversary encryptor can query the hash function $H(\cdot)$ on $\ell^2 \log(\ell)$ distinct identity tags, find $\ell \log(\ell)$ identity tags that collide at the same $\log(\ell)$ bit vectors $\tilde{h}_i$, and submit ciphertexts created using these identity tags and bit vectors. Then, the block proposer must publish $O(\ell/\log(\ell))$ pre-decryption keys to enable the decryption of this batch of ciphertexts.

6 Batch Decryption from Attribute Based Encryption

In this section, we present a batch decryption system that uses, as a black-box, a KP-ABE scheme with attribute space $\mathcal{I} = \{0, 1\}^k$ and message space $\mathcal{M}$, instantiated with the following class of functions:

$$\mathcal{F}_{k,\ell} = \{f_S: \{0, 1\}^k \rightarrow \{0, 1\} \mid S \subseteq (\{0, 1\}^k)^\ell, |S| = \ell, f_S(x) = 0 \text{ if and only if } x \in S\}.$$

6.1 Definitions

Definition 10. A key-policy attribute-based encryption (KP-ABE) scheme for some class of functions $\mathcal{F}$ and message space $\mathcal{M}$ consists of four algorithms:

- $\text{ABE.SETUP}(1^\lambda, \mathcal{F}) \rightarrow (\text{mpk}, \text{msk})$: The setup algorithm takes as input the security parameter λ and the description of the class of functions $\mathcal{F}$, and outputs the master public key mpk and the master secret key msk .
- $\text{ABE.ENC}(\text{mpk}, x, \text{m}) \rightarrow \text{ct}$: The encryption algorithm takes as input mpk , an attribute $x \in \{0, 1\}^*$ and a message $\text{m} \in \mathcal{M}$, and outputs an attribute-ciphertext tuple (x, ct) .
- $\text{ABE.KEYGEN}(\text{msk}, f) \rightarrow \text{sk}_f$: The key generation algorithm takes as input msk and a function $f \in \mathcal{F}$, and outputs a secret key sk_f .
- $\text{ABE.DEC}(\text{sk}_f, (x, \text{ct})) \rightarrow \text{m}$: The decryption algorithm takes as input a secret key sk_f and an attribute-ciphertext tuple (x, ct) , and if $f(x) = 0$, outputs the corresponding message m .

Definition 11 (Correctness of KP-ABE, taken from [2]). For every function $f \in \mathcal{F}$, attribute strings $x \in \{0, 1\}^*$, where $f(x) = 0$, and messages $\text{m} \in \mathcal{M}$, it holds that

$$\Pr \left[\text{ABE.DEC}(\text{sk}_f, (x, \text{ct})) = \text{m} \mid \begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{ABE.SETUP}(1^\lambda, \mathcal{F}) \\ \text{ct} \leftarrow \text{ABE.ENC}(\text{mpk}, x, \text{m}) \\ \text{sk}_f \leftarrow \text{ABE.KEYGEN}(\text{msk}, f) \end{array} \right] > 1 - \text{negl}(\lambda)$$

Definition 12 (Selective Security of KP-ABE, taken from [2]). An ABE scheme for a class of functions $\mathcal{F}_\lambda = \{f : \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda\}$ is selectively secure if for all PPT adversaries $\mathcal{A}$, where $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$, it holds that

$$|\Pr[\text{Game}_{\mathcal{A},0}^{\text{ABE}}(\lambda) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{ABE}}(\lambda) = 1]| \leq \text{negl}(\lambda),$$

where for each $b \in \{0, 1\}$ and $\lambda \in \mathbb{N}$, the game $\text{Game}_{\mathcal{A},b}^{\text{ABE}}(\lambda)$ is defined as follows:

1. $(x^* \in \mathcal{X}_\lambda, \text{state}_1) \leftarrow \mathcal{A}_1(\lambda)$, // $\mathcal{A}$ commits to the challenge attribute x^* .
2. $(\text{mpk}, \text{msk}) \leftarrow \text{ABE.SETUP}(\lambda)$
3. $(\text{m}_0, \text{m}_1, \text{state}_2) \leftarrow \mathcal{A}_2^{\text{KG}(\text{msk}, x^*, \cdot)}(\text{mpk}, \text{state}_1)$ // $\mathcal{A}$ outputs messages.
4. $\text{ct}^* \leftarrow \text{ABE.ENC}(\text{mpk}, x^*, \text{m}_b)$
5. $b' \leftarrow \mathcal{A}_3^{\text{KG}(\text{msk}, x^*, \cdot)}(\text{ct}^*, \text{state}_2)$ // $\mathcal{A}$ outputs a guess b' for b
6. Output $b' \in \{0, 1\}$,

where $\text{KG}(\text{msk}, x^*, f)$ returns a secret key $\text{sk}_f = \text{ABE.KEYGEN}(\text{msk}, f)$ if $f(x^*) \neq 0$ and $\perp$ otherwise.

A fully secure ABE scheme has a similar definition, except that the adversary determines the challenge attribute x^* after receiving mpk and making a polynomial number of secret key queries. Any selectively secure ABE scheme can be converted into a fully secure one at the cost of an exponential loss in the parameter size [8].

6.2 The Construction

BD.Setup (1^λ) . The setup algorithm invokes

$$\text{ABE.SETUP}(1^\lambda, \mathcal{F}_{k,\ell}) \rightarrow (\text{ABE.mpk}, \text{ABE.msk})$$

and outputs $\text{BD.pk} := \text{ABE.mpk}$ and $\text{BD.sk} := \text{ABE.msk}$.

BD.Enc(**BD.pk**, $r \in \{0, 1\}^k$, $m \in \mathcal{M}$). Parsing $\text{ABE.mpk} := \text{BD.pk}$, the encryption algorithm calls

$$\text{ABE.ENC}(\text{ABE.mpk}, r, m) \rightarrow \text{ABE.ct}.$$

and outputs $\text{ct} := (r, \text{ABE.ct})$.

BD.PreDec(**BD.sk**, $(r_i)_{i \in [\ell]}$). Parsing $\text{ABE.msk} := \text{BD.sk}$, the pre-decryption algorithm invokes

$$\text{ABE.KEYGEN}(\text{ABE.msk}, f) \rightarrow \text{ABE.sk}_f,$$

with the function $f: \{0, 1\}^k \rightarrow \{0, 1\}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i \in [\ell]}$. It outputs $\text{BD.sbk} := \text{ABE.sk}_f$.

BD.Dec(**BD.pk**, **BD.sbk**, $(\text{ct}_i)_{i \in [\ell]}$). The decryption algorithm parses $\text{ABE.pk} := \text{BD.pk}$, $\text{ABE.sk}_f := \text{BD.sbk}$, and $(r_i, \text{ABE.ct}_i) := \text{ct}_i$. Then, for each $i \in [\ell]$, it outputs

$$\text{ABE.DEC}(\text{sk}_f, (r_i, \text{ABE.ct}_i)) \rightarrow m_i.$$

6.3 Correctness

For $(\text{BD.pk}, \text{BD.sk}) := (\text{ABE.mpk}, \text{ABE.msk}) \leftarrow \text{ABE.SETUP}(1^\lambda, \mathcal{F})$, correct ciphertexts $\text{ct}_i := (r_i \in \{0, 1\}^k, \text{ABE.ct}_i)$, where $\text{ABE.ct}_i \leftarrow \text{ABE.ENC}(\text{ABE.mpk}, r_i, m_i)$ for $i \in [\ell]$, and a correct pre-decryption key $\text{BD.sbk} := \text{sk}_f \leftarrow \text{ABE.KEYGEN}(\text{ABE.msk}, f)$ for the function $f \in \mathcal{F}_{k,l}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i \in [\ell]}$, by the correctness of the KP-ABE scheme,

$$\text{BD.DEC}(\text{BD.pk}, \text{BD.sbk}, (\text{ct}_i)_{i \in [\ell]}) = (\text{ABE.DEC}(\text{sk}_f, (r_i, \text{ABE.ct}_i)))_{i \in [\ell]} = (m_i)_{i \in [\ell]}$$

except with negligible probability.

6.4 Security

Theorem 4. *The BD scheme instantiated with a selectively secure ABE protocol is selectively secure (as in [Definition 3](#)).*

Proof. Suppose that a PPT adversary $\mathcal{A}$ has a non-negligible advantage ϵ against the BD game of [Definition 3](#). Then, we construct an adversary $\mathcal{B}$ with advantage at least ϵ in the ABE security game. Here, $\mathcal{B}$ simulates the view of $\mathcal{A}$ as follows:

- $\mathcal{B}$ receives a challenge identity r^* from $\mathcal{A}$ and sends the challenge attribute $x^* := r^*$ to Ch.
- $\mathcal{B}$ receives ABE.mpk for the class of functions $\mathcal{F}_{k,l}$ from the challenger Ch and sends $\text{BD.pk} := \text{ABE.mpk}$ to $\mathcal{A}$.
- Whenever $\mathcal{A}$ sends a pre-challenge query for the identities $(r_1, \dots, r_\ell)$ that does not contain r^* , $\mathcal{B}$ queries the oracle $\text{KG}(\text{ABE.msk}, x^*, \cdot)$ with the function $f: \{0, 1\}^k \rightarrow \{0, 1\}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i \in [\ell]}$. Upon receiving the secret key sk_f , it forwards $\text{BD.sbk} := \text{sk}_f$ to $\mathcal{A}$. (If $r^* \in (r_1, \dots, r_\ell)$, then $\mathcal{B}$ instead aborts and returns a random bit.)
- When $\mathcal{A}$ sends the two messages (m_0, m_1) it would like to be challenged on, $\mathcal{B}$ forwards m_0, m_1 to Ch. Upon receiving a ciphertext ct^* from Ch, it forwards (r^*, ct^*) to $\mathcal{A}$.
- For the post-challenge queries, $\mathcal{B}$ repeats the same approach as the pre-challenge queries.
- Once $\mathcal{A}$ outputs a bit b' , $\mathcal{B}$ outputs the same bit.

We next analyze the reduction. Conditioned on $\mathcal{B}$ not aborting, $\text{KG}(\text{ABE.msk}, r^*, \cdot)$ responds with the correct secret key $\text{sk}_f \leftarrow \text{ABE.KEYGEN}(\text{ABE.msk}, f)$, which is the same as the pre-decryption key BD.sbk corresponding to the sequence of identities defining the function f . Hence, the view $\mathcal{B}$ provides to $\mathcal{A}$ is statistically indistinguishable from the view of a real interaction with the challenger of the BD scheme. Hence, $\mathcal{A}$ has advantage ϵ against the BD game of Definition 3. Now, when $\mathcal{A}$ wins the BD game in Definition 3, $\mathcal{B}$ wins the ABE game in Definition 12. Therefore, $\mathcal{B}$ has advantage at least ϵ against the ABE game of Definition 12. $\square$

6.5 Parameter Sizes

We next estimate the public key, ciphertext and pre-decryption key sizes achieved by the BD systems instantiated with three LWE-based KP-ABE schemes chosen due to their succinct secret keys: BGG+14 [14], CW23 [22], and Wee24 [46].

The KP-ABE scheme by BGG+14. The KP-ABE scheme by BGG+14 supports boolean circuits with inputs in $\{0, 1\}$, as well as arithmetic circuits with inputs in $\mathbb{Z}_q$. To support circuits of depth d and w -way multiplication gates, where all but one of the inputs are in $[-p, p]$, the protocol chooses n such that $n \geq d^{1/\epsilon}(2 \log_2(n) + w \log(p))^{1/\epsilon}$, sets $q = 2^{n^\epsilon}$ and $m = O(n \log q)$, where ϵ links the LWE problem to the problem of approximating the worst-case GapSVP to $\tilde{O}(n/2^{n^\epsilon})$ factors.

We can express the functions $f_S(x) = \prod_{i \in [\ell]} (x - r_i)$, where $x, r_i \in \{0, 1\}^k$, as boolean circuits with k -bit inputs as follows. Let $x^j, j \in [k]$, denote the bits of the input identity x , and $r_i^j, j \in [k]$, denote the bits of the identity r_i . Let $\neg, \vee$ and $\wedge$ denote the ‘complement’, ‘or’, and ‘and’ operations respectively. Then, the circuit computes

$$\neg \left(\bigvee_{i=1}^{\ell} \left(\bigwedge_{j=1}^k (x^j \oplus r_i^j) \right) \right)$$

Note that this circuit can be expressed as depth $O(\log(k\ell)) = O(\log(k) + \log(\ell)) = O(\log(\ell))$ boolean and arithmetic circuits with k -bit inputs, where the fan-in is 2, *i.e.*, $w = 2$, and each input is a bit, *i.e.*, $p = 2$ (we assume that $k \leq \ell$ for the KP-ABE schemes). Finally, this implies $n = O(\log^{1/\epsilon}(k))$.

Now, the KP-ABE secret keys are low-norm matrices in $\mathbb{Z}^{2m \times m}$, sampled from a Gaussian distribution with width $\sigma = \omega(\sqrt{n} \log(m))$. Then, given $\log(q) = n^\epsilon = O(\log(\ell))$, the public keys scale as

$$|\text{pp}| = O(knm \log(q)) = O(kn^2 \log^2(q)) = O(k\lambda^2 \log^2(\ell)),$$

the pre-decryption keys scale as

$$|\text{sbk}| = O(m^2 \log(\sigma)) = O(n^2 \log^2(q) \log(n)) = O(\lambda^2 \log^2(\ell)),$$

and ciphertexts scale as

$$|\text{ct}| = O(km \log(q)) = O(kn \log^2(q)) = O(k\lambda \log^2(\ell)).$$

We ignore the ϵ terms in the power; otherwise, the power of $\log(\ell)$ would be $2 + 2/\epsilon$.

The KP-ABE scheme by CW23. The KP-ABE scheme by CW23 supports boolean circuits with inputs in $\{0, 1\}^k$ and achieve keys of size $O(\text{poly}(\lambda))$, independent of the circuit depth. More specifically, CW23 achieves parameters of size

$$\begin{aligned} |\text{mpk}| &= O(kn_1^2 \log^2(q)) \\ |\text{sk}| &= O(n_0 \lambda \log^2(p)) \\ |\text{ct}| &= O(kn_1 \log^2(q)) \end{aligned}$$

where $n_0 = O(\lambda)$, $n_1 = O(\lambda d^{1/\epsilon})$ and $\log(q) = O(d\lambda)$ are analogous to the parameters n and $\log(q)$ in BGG+14, d is the circuit depth, and λ is the security parameter. Note that $\log(p) = O(\lambda)$ has no dependence on the circuit depth.

Now, as in BGG+14, we can express the functions $f_S(x) = \prod_{i \in [\ell]} (x - r_i)$, where $x, r_i \in \{0, 1\}^k$, as depth $O(\log(\ell))$ boolean circuits with k -bit inputs, where the fan-in is 2, *i.e.*, $w = 2$, and each input is a bit, *i.e.*, $p = 2$. Therefore, the batch decryption scheme based on CW23 achieves parameters of size

$$\begin{aligned} |\text{pk}| &= O(k\lambda^4 \log^2(\ell)) \\ |\text{sbk}| &= O(\lambda^4) \\ |\text{ct}| &= O(k\lambda^3 \log^2(\ell)) \end{aligned}$$

We again ignore the $2/\epsilon$ term in the power.

The KP-ABE scheme by Wee24. Using ℓ -succinct LWE, Wee24 can achieve any $|\text{pk}|$ and $|\text{ct}|$ size of the form $O(k^{2\alpha} \text{poly}(\lambda, d))$ and $k + O(k^{1-\alpha} \text{poly}(\lambda, d))$, where $\alpha \in [1/3, 1]$.

Setting $k = \beta \log(\ell)$, the public key and ciphertext sizes are comparable to those of our ‘vanilla’ batch decryption scheme (Section 5) without the sub-batches technique (SB) in [18, §5.3] or the solution (SB*) in Section 5.4 (see Table 1). However, the BGG+14 construction has quadratically larger pre-decryption keys; and neither of the techniques in [18, §5.3] and Section 5.4 (SB and SB*) help the KP-ABE based constructions avoid the linear dependency of their public key and ciphertext size on k . Therefore, the KP-ABE based constructions have larger public keys and ciphertexts compared to our final batch decryption scheme.

7 Batch Decryption from Trilinear Maps

In this section, we present a batch decryption scheme based on trilinear maps.

7.1 Background on Trilinear Maps and Generic Group Model

Let the product notation $(\cdot) : \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_3 \rightarrow \mathbb{G}_t$ denote a trilinear map, where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$ and $\mathbb{G}_t$ are multiplicative groups of prime order p with generators G_1, G_2, G_3 , and $G_T = G_1 \cdot G_2 \cdot G_3$, respectively. Let $H : \mathbb{G}_t \rightarrow \{0, 1\}^{|\mathcal{M}|}$ denote a one-way function.

The following description of Shoup’s GGM [41] is adapted from [48, 1]. Let $S \subseteq \{0, 1\}^*$ be a set of strings with size at least $p \in \mathbb{Z}^+$ and with a known upper bound. A group $\mathbb{G}$ of prime order p is generically represented by sampling a random injection $L : \mathbb{Z}_p \rightarrow S$, known as the labeling function. This function encodes the string representation of xG , where $G \in \mathbb{G}$ is a fixed generator

of $\mathbb{G}$. In the following sections, L will be generated dynamically as new queries arrive, and we will treat it as a dictionary indexed by values from its domain. The model provides the following query functionalities:

- **Labeling queries:** Given $x \in \mathbb{Z}_p$, returns $L(x)$.
- **Group operations:** Given $(\ell_1, \ell_2, a_1, a_2) \in S^2 \times \mathbb{Z}_p^2$, if there are $x_1, x_2 \in \mathbb{Z}_p$ such that $L(x_1) = \ell_1$ and $L(x_2) = \ell_2$, returns $L(a_1 x_1 + a_2 x_2)$. Otherwise, returns $\perp$.

In the trilinear group setting, the model also supports an additional query type:

- **Mapping query:** Given $(\ell_1, \ell_2, \ell_3) \in S_1 \times S_2 \times S_3$, if there are $x_1, x_2, x_3 \in \mathbb{Z}_p$ such that $L_1(x_1) = \ell_1$, $L_2(x_2) = \ell_2$, and $L_3(x_3) = \ell_3$, returns $L_T(x_1 \cdot x_2 \cdot x_3)$. Otherwise, returns $\perp$.

7.2 The Construction

BD.Setup(1^λ). The setup algorithm samples $\alpha \leftarrow \mathbb{Z}_p$ and outputs the secret key $\text{BD.sk} := \alpha$. It calculates $A_1 := \alpha G_1$, and $B_i := \alpha^i G_2$ for $i \in [\ell - 1]$, and outputs the public key $\text{BD.pk} := (A_1, B_1, \dots, B_{\ell-1})$.

BD.Enc($\text{BD.pk}, r \in \mathbb{Z}_p, m \in \{0, 1\}^m$). Parsing $(A_1, B_1, \dots, B_{\ell-1}) := \text{BD.pk}$, the encryption algorithm samples $t \leftarrow \mathbb{Z}_p$ and outputs $\text{ct} := (r, \text{ct}[1], \text{ct}[2])$:

$$\text{ct}[1] := tA_1 + rtG_1 \quad \text{ct}[2] := m \oplus H(tG_T)$$

BD.PreDec($\text{BD.sk}, (r_i)_{i \in [\ell]}$). Parsing $\alpha := \text{BD.sk}$, the pre-decryption algorithm outputs

$$\text{BD.sbk} := \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3.$$

BD.Dec($\text{BD.pk}, \text{BD.sbk}, (\text{ct}_i)_{i \in [\ell]}$). After parsing the public key and ciphertexts $(r_i, \text{ct}_i[1], \text{ct}_i[2]) := \text{ct}_i$, for each $i \in [\ell]$, the decryption algorithm finds the coefficients $(f_i^j)_{j=0, \dots, \ell-2}$, of the polynomials

$$f_i(X) = X^{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j X^j = \prod_{j \in [\ell], j \neq i} (X + r_j).$$

Then, for each $i \in [\ell]$, it calculates

$$f_i(\alpha)G_2 = \prod_{j \in [\ell], j \neq i} (\alpha + r_j)G_2 = B_{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j B_j,$$

where $B_0 = G_2$. Finally, for $i \in [\ell]$, it outputs

$$m'_i := \text{ct}_i[2] \oplus H(\text{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BD.sbk}).$$

7.3 Correctness

For a correct ciphertext $\text{ct}_i := (r_i, \text{ct}_i[1], \text{ct}_i[2])$, where $\text{ct}_i[1] := tA_1 + r_itG_1$ and $\text{ct}_i[2] := m_i \oplus H(tG_T)$, and correct pre-decryption key $\text{sbk} := \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3$, it holds that

$$\begin{aligned} m'_i &= \text{ct}_i[2] \oplus H(\text{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BD.sbk}) \\ &= m_i \oplus H(tG_T) \oplus H\left((\alpha + r_i)tG_1 \cdot \prod_{j \in [\ell], j \neq i} (\alpha + r_j)G_2 \cdot \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3\right) \\ &= m_i \oplus H(tG_T) \oplus H(tG_T) = m_i \end{aligned}$$

7.4 Thresholdizing the Secret Key

To thresholdize the pre-decryption key $\frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3$, where $r_1, \dots, r_\ell$ are all distinct, we can first express it as a sum of partial fractions:

$$\frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} G_3 = \sum_{i \in [\ell]} \frac{\beta_i}{\alpha + r_i} G_3$$

for some $\beta_1, \dots, \beta_\ell \in \mathbb{Z}_p$. Now, suppose α is t -out-of- n secret shared among n parties. Then for $i \in [\ell]$, it suffices to describe a secure protocol for computing $(1/(\alpha + r_i))G_3$ among a quorum of t parties. This problem is the same as computing a threshold signature in the Boneh-Boyen signature scheme [7], and Wang et al. [43] give an efficient two-round protocol for this problem. Briefly, the parties first jointly compute and publish $\beta = s(\alpha + r_i) \in \mathbb{Z}_p$ in the clear, where $s \leftarrow \mathbb{Z}_p$ is additively secret shared among them, say $s = s_1 + \dots + s_t$. Each party then publishes $P_i = (s_i/\beta)G_3$. Then, $\sum_{i \in [t]} P_i = (1/(\alpha + r_i))G_3$, as required.

7.5 Security

Theorem 5. *For all $\ell \in \mathbb{N}$ and unbounded adversaries $\mathcal{A}$ making at most q queries to the group oracle, hash oracle, and for key computation, the BD scheme is selectively secure (Definition 3) in the generic group model.*

For the proof of Theorem 5, we follow the GGM proof blueprint and notation by Agarwal, Fernando, Pinkas [1], applied to our protocol. The challenger simulates the GGM oracle by selecting secret integers $x_1, \dots, x_n \in \mathbb{Z}_p$ and responding to the adversary's queries with $L(f(x_1, \dots, x_n))$ for some functions f . In the security proof, $x_i \in \mathbb{Z}_p$ are replaced with indeterminates X_i as in the symbolic model of Shoup [41], and the challenger reveals $L(f(X_1, \dots, X_n))$ to the adversary. Before we present the proof, in Figure 2 and Figure 3, we describe the security game in the GGM and symbolic model respectively.

Proof of Theorem 5 relies on the following lemmas:

Lemma 23. *For all $\ell \in \mathbb{N}$ and all unbounded adversaries $\mathcal{A}$,*

$$\left| \Pr[\text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| = 0.$$

Security Game in the Generic Group Model: $\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$

Setup: Let $S_1 = S_2 = S_3 = S_T = \{0,1\}^{p'}$ where $2^{p'} \geq p$, and p is the group order. The challenger first initializes the labeling functions $L_i : \mathbb{Z}_p \rightarrow S_i$, $i \in \{1, 2, 3, T\}$, and the hash function $H : S_T \rightarrow \{0,1\}^m$. It then receives the challenge identity r^* , and runs $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{BD.pk}, \text{BD.sk})$. During the call to BD.SETUP , it does the following:

- Sample $\alpha \leftarrow \mathbb{Z}_p$.
- Sample $e_\alpha^1 \leftarrow S_1$, and set $L_1(\alpha) := e_\alpha^1$ and $S_1 := S_1 \setminus \{e_\alpha^1\}$.
- For $i \in [\ell - 1]$, sample $e_{\alpha^i}^2 \leftarrow S_2$, and set $L_2(\alpha^i) := e_{\alpha^i}^2$ and $S_2 := S_2 \setminus e_{\alpha^i}^2$.

It sets $\text{BD.pk} := (L_1(\alpha), L_2(\alpha), \dots, L_2(\alpha^{\ell-1}))$ and sends BD.pk to $\mathcal{A}$.

Group and hash oracle queries:

- **Labeling query:** For a group $\mathbb{G}_i$, upon receiving $x \in \mathbb{Z}_p$, if $x \notin L_i$, Ch samples $e \leftarrow S_i$, sets $L_i(x) := e$ and updates $S_i := S_i \setminus \{e\}$. It then sends $L_i(x)$ to $\mathcal{A}$.
- **Group operation:** Upon receiving $(e_1, e_2, a_1, a_2) \in (\{0,1\}^{p'})^2 \times \mathbb{Z}_p^2$, if there are $x_1, x_2 \in \mathbb{Z}_p$ such that $L_i(x_1) = e_1$ and $L_i(x_2) = e_2$, Ch computes $x_3 := a_1 x_1 + a_2 x_2$, does a labeling query for x_3 , and sends $L_i(x_3)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Map operation:** Upon receiving (e_1, e_2, e_3) , if there are $x_1, x_2, x_3 \in \mathbb{Z}_p$ such that $L_1(x_1) = e_1$, $L_2(x_2) = e_2$ and $L_3(x_3) := e_3$, Ch computes $x_T = x_1 \cdot x_2 \cdot x_3$, does a labeling query for x_T , and sends $L_T(x_T)$ to $\mathcal{A}$.
- **Hash query:** Upon receiving e , if there is $x \in \mathbb{Z}_p$ such that $L_T(x) = e$, and $e \notin H$, Ch samples $h \leftarrow \mathbb{Z}_{p'}$, sets $H(e) := h$, and sends h to $\mathcal{A}$.

Pre and post-challenge queries: Upon receiving a list of ℓ identities $(r_1, \dots, r_\ell) \subseteq \mathcal{I}^\ell$ from $\mathcal{A}$, Ch does the following:

- If the challenge identity $r^* \in (r_1, \dots, r_\ell)$, it halts the game.
- If $x := \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} \notin L_3$, it does a labeling query for x .
- It sends the pre-decryption key $\text{BD.sbk} := L_3(x)$ to $\mathcal{A}$.

Challenge round: Upon receiving the messages $m_0, m_1 \in \mathbb{Z}_{p'}$ from $\mathcal{A}$, Ch does the following:

- It samples $t^* \leftarrow \mathbb{Z}_q$ and does a group operation $(L_1(\alpha), G_1, t^*, r^* t^*)$ for L_1 with output set to $\text{ct}^*[1]$.
- It does a group operation $(1, G_T, 1, t^*)$ for L_T with output e . It then does a hash query for e with output mask . It sets $\text{ct}^*[2]$ to be $m_b \oplus \text{mask}$.
- It sends the ciphertext $\text{ct}^* = (r^*, \text{ct}^*[1], \text{ct}^*[2])$ to $\mathcal{A}$.

Output: Once $\mathcal{A}$ halts and outputs a bit b' , the game outputs b' .

Fig. 2: The security game for batch decryption in the generic group model.

Security Game in the Symbolic Model: $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell)$

Setup: Let $S_1 = S_2 = S_3 = S_T = \{0,1\}^{p'}$ where $2^{p'} \geq p$, and p is the group order. The challenger first initializes the labeling functions $L_i : \mathbb{Z}_p \rightarrow S_i$, $i \in \{1, 2, 3, T\}$, and the hash function $H : S_T \rightarrow \{0,1\}^m$. It then receives the challenge identity r^* and runs $\text{BD.SETUP}(1^\lambda, 1^\ell) \rightarrow (\text{BD.pk}, \text{BD.sk})$. During the call to BD.SETUP , it does the following:

- Sample $e_\alpha^1 \leftarrow S_1$, and set $L_1(X_\alpha) := e_\alpha^1$ and $S_1 := S_1 \setminus \{e_\alpha^1\}$, where X_α is an indeterminate corresponding to BD.sk .
- For $i \in [\ell - 1]$, sample $e_{\alpha^i}^2 \leftarrow S_2$, and set $L_2(X_\alpha^i) := e_{\alpha^i}^2$ and $S_2 := S_2 \setminus e_{\alpha^i}^2$.

It sets $\text{BD.pk} := (L_1(X_\alpha), L_2(X_\alpha), \dots, L_2(X_\alpha^{\ell-1}))$ and sends BD.pk to $\mathcal{A}$.

Group and hash oracle queries:

- **Labeling query:** For a group $\mathbb{G}_i$, upon receiving $x \in \mathbb{Z}_p$, if $x \notin L_i$, Ch samples $e \leftarrow S_i$, sets $L_i(x) := e$ and updates $S_i := S_i \setminus \{e\}$. It then sends $L_i(x)$ to $\mathcal{A}$.
- **Group operation:** Upon receiving $(e_1, e_2, a_1, a_2) \in (\{0,1\}^{p'})^2 \times \mathbb{Z}_p^2$, if there are polynomials $X_1, X_2 \in \mathbb{Z}_p[*]$ such that $L_i(X_1) = e_1$ and $L_i(X_2) = e_2$, Ch computes the polynomial $X_3 := a_1 X_1 + a_2 X_2$, does a labeling query for X_3 , and sends $L_i(X_3)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Map operation:** Upon receiving (e_1, e_2, e_3) , if there are polynomials $X_1, X_2, X_3 \in \mathbb{Z}_p[*]$ such that $L_1(X_1) = e_1$, $L_2(X_2) = e_2$ and $L_3(X_3) = e_3$, Ch computes the polynomial $X_T := X_1 \cdot X_2 \cdot X_3$, does a labeling query for X_T , and sends $L_T(X_T)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Hash query:** Upon receiving e , if there is a polynomial $X \in \mathbb{Z}_p[*]$ such that $L_T(X) = e$, and $e \notin H$, Ch samples $h \leftarrow \mathbb{Z}_{p'}$, sets $H(e) := h$, and sends h to $\mathcal{A}$.

Pre and post-challenge queries: Upon receiving a list of ℓ identities $(r_1, \dots, r_\ell) \subseteq \mathcal{I}^\ell$ from $\mathcal{A}$, Ch does the following:

- If the challenge identity $r^* \in (r_1, \dots, r_\ell)$, it halts the game.
- Let $X_{\text{sbk}} := \frac{1}{\prod_{i \in [\ell]} (X_\alpha + r_i)}$ be a Laurent polynomial with the indeterminate X_α . If $X_{\text{sbk}} \notin L_3$, Ch does a labeling query for X_{sbk} .
- It sends the pre-decryption key $\text{BD.sbk} := L_3(X_{\text{sbk}})$ to $\mathcal{A}$.

Challenge round: Upon receiving the messages $m_0, m_1 \in \mathbb{Z}_{p'}$ from $\mathcal{A}$, Ch does the following:

- Let X_{t^*} be an indeterminate. Ch sets $L_1(X_{t^*} X_\alpha + r^* X_{t^*}) \leftarrow \{0,1\}^{p'}$ and $\text{ct}^*[1] := L_1(X_{t^*} X_\alpha + r^* X_{t^*})$.
- It then sets $L_T(X_{t^*}) \leftarrow \{0,1\}^{p'}$ and does a hash query for $L_T(X_{t^*})$ with output mask . It sets $\text{ct}^*[2] = m_b \oplus \text{mask}$.
- It sends the ciphertext $\text{ct}^* = (r^*, \text{ct}^*[1], \text{ct}^*[2])$ to $\mathcal{A}$.

Output: Once $\mathcal{A}$ halts and outputs a bit b' , the game outputs b' .

Fig. 3: The security game for batch decryption in the symbolic model.

Proof. We describe an intermediate hybrid game $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell)$ such that $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell) \equiv \text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell)$ for $b \in \{0, 1\}$, which implies the lemma. Here, $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}(1^\lambda, 1^\ell)$ is the same as $\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell)$ except that $\text{ct}^*[2]$ is computed as follows:

- The challenger defines an indeterminate X_u , sets $L_T(X_u) \leftarrow S_T$ and updates $S_T := S_T \setminus L_T(u)$.
- It does a hash query for $L_T(X_u)$ with output mask . It sets $\text{ct}^*[2] = \mathbf{m}_b \oplus \text{mask}$.

Suppose the adversary makes n queries for the pre-decryption key with the identity tags r_i , $i \in [n\ell]$. In this case, it holds the following multivariate Laurent polynomials:

$$\begin{aligned} L_1 &:= \{1, X_\alpha, X_{t^*}(X_\alpha + r^*)\} \\ L_2 &:= \{X_\alpha^i\}_{i=0,\dots,\ell-1} \\ L_3 &:= \left\{1, \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)}\right\}_{j=0,\dots,n-1} \end{aligned}$$

The polynomial used to calculate $\text{ct}^*[2]$ in $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}$ is X_{t^*} (for both $b = 0, 1$), and in order to prove the equivalence of the adversary's views in $\text{Game}_{\mathcal{A}}^{\text{Hyb,SM}}$ and $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}$, we argue that X_{t^*} is linearly independent of the polynomials in $L_1 \otimes L_2 \otimes L_3$. Towards contradiction, suppose this is not the case. Since X_{t^*} appears only as part of the monomials $X_{t^*}(X_\alpha + r^*)$, it must be that

$$X_{t^*} = X_{t^*}(X_\alpha + r^*) \sum_{j=0}^{n-1} f_j(X_\alpha) \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)},$$

where $f_j(\cdot)$ are degree $\leq (\ell - 1)$ polynomials in X_α . In other words,

$$\frac{1}{(X_\alpha + r^*)} = \sum_{j=0}^{n-1} f_j(X_\alpha) \frac{1}{\prod_{i \in [j\ell+1, (j+1)\ell]} (X_\alpha + r_i)}.$$

Now, for this equality to be satisfied, r^* must be among the values $(r_i)_{i \in [n\ell]}$, as otherwise $f_j(X_\alpha)$ would have to be a rational polynomial. However, this is a contradiction as $r^* \notin (r_{j\ell+1}, \dots, r_{(j+1)\ell})$ for any $j = 0, \dots, n-1$. $\square$

Lemma 24. *For all $\ell \in \mathbb{N}$ and unbounded $\mathcal{A}$ making at most q queries to the group oracle, hash oracle, and for key computation, for $b \in \{0, 1\}$;*

$$\left| \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| \leq \binom{q + \ell + 3}{2} \frac{\ell}{p}.$$

Proof. Note that the challenger can create a perfect simulation of the game $\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$ by sampling values from $\mathbb{Z}_p$ uniformly at random at the end of the game $\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell)$ for the involved indeterminates, unless the sampled values cause different polynomials to assume the same value. By the Schwartz-Zippel lemma [49,39], the probability of this event for two polynomials of degree (d_1, d_2) is at most $\max(d_1, d_2)/p$. Now, when the adversary makes q pre-decryption queries, there will be q Laurent polynomials from the pre-decryption queries, $\ell + 1$ polynomials from setup (including the '1' polynomials) and 2 polynomials from encryption, implying a total of $q + \ell + 3$

polynomials across all four groups. Since the largest degree across the polynomials is ℓ , by the union bound, the probability that different polynomials assume the same value can be upper-bounded by

$$\binom{q + \ell + 3}{2} \frac{\ell}{p}.$$

□

Proof of Theorem 5. Since by Lemma 23,

$$\left| \Pr[\text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| = 0,$$

by Lemma 24,

$$\left| \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\text{Game}_{\mathcal{A},b}^{\text{BD,SM}}(1^\lambda, 1^\ell) = 1] \right| \leq \binom{q + \ell + 3}{2} \frac{\ell}{p},$$

and

$$\binom{q + \ell + 3}{2} \frac{\ell}{p} = \text{negl}(\lambda),$$

it holds that

$$\text{Game}_{\mathcal{A},0}^{\text{BD,GGM}}(1^\lambda, 1^\ell) \approx_c \text{Game}_{\mathcal{A},0}^{\text{BD,SM}}(1^\lambda, 1^\ell) \equiv \text{Game}_{\mathcal{A},1}^{\text{BD,SM}}(1^\lambda, 1^\ell) \approx_c \text{Game}_{\mathcal{A},1}^{\text{BD,GGM}}(1^\lambda, 1^\ell)$$

□

8 Conclusions and open problems

We have presented three new batch decryption and batch threshold decryption systems without epochs, based respectively on key-policy ABE, lattices, and trilinear maps. Our novel lattice-based batch decryption scheme is plausibly post-quantum secure, has shorter public keys and ciphertexts compared to the KP-ABE schemes and uses techniques from the ABB10 identity-based encryption [2] and the WWW25 preimage sampler [45]. It can be thresholdized by secret sharing the trapdoor for a random matrix employed by the construction through the technique from [6, §3]. Although this enables keeping the parameter, key and ciphertext sizes of our scheme succinct in the number of parties, it necessitates two rounds of interaction. Designing non-interactive thresholding techniques for lattice-based batch decryption schemes remains an open problem.

Acknowledgments. We thank Gregory Neven for helpful discussions about this work. This work was partially supported by the Simons Foundation, the Stanford Center for Blockchain Research, and UBRI. The third author was further supported by a Research Hub Collaboration agreement between Stanford University and Input Output Global Inc.

References

1. Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. 2024. To appear in CRYPTO 2025.
2. Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, volume 6110 of *Lecture Notes in Computer Science*, pages 553–572. Springer, 2010.
3. Noga Alon, Eyal Lubetzky, and Ori Gurel-Gurevich. Choice-memory tradeoff in allocations. In *FOCS*, pages 230–238. IEEE Computer Society, 2009.
4. Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898, 2022.
5. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In *TCC*, volume 5978 of *Lecture Notes in Computer Science*, pages 201–218. Springer, 2010.
6. Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *Lecture Notes in Computer Science*, pages 218–236. Springer, 2013.
7. Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer, 2004.
8. Dan Boneh and Xavier Boyen. Efficient selective identity-based encryption without random oracles. *J. Cryptol.*, 24(4):659–693, 2011.
9. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In *CT-RSA*, volume 3860 of *Lecture Notes in Computer Science*, pages 226–243. Springer, 2006.
10. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. Cryptology ePrint Archive, Paper 2025/279, 2025.
11. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. Cryptology ePrint Archive, Paper 2025/279, 2025.
12. Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. *SIAM J. Comput.*, 36(5):1301–1328, 2007.
13. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer, 2001.
14. Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In *EUROCRYPT*, volume 8441 of *Lecture Notes in Computer Science*, pages 533–556. Springer, 2014.
15. Dan Boneh, Craig Gentry, and Michael Hamburg. Space-efficient identity based encryption without pairings. In *FOCS*, pages 647–657. IEEE Computer Society, 2007.
16. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *CRYPTO (1)*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428. Springer, 2013.
17. Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2008.
18. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. BEAT-MEV: epochless approach to batched threshold encryption for MEV prevention. *IACR Cryptol. ePrint Arch.*, page 1533, 2024. To appear in Usenix Security 2025.
19. Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. *IACR Cryptol. ePrint Arch.*, page 44, 2025.
20. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *USENIX Security Symposium*. USENIX Association, 2024.
21. Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. *IACR Cryptol. ePrint Arch.*, page 1516, 2024. To appear in Usenix Security 2025.
22. Valerio Cini and Hoeteck Wee. ABE for circuits with poly(λ)-sized keys from LWE. In *FOCS*, pages 435–446. IEEE, 2023.
23. Clifford C. Cocks. An identity based encryption scheme based on quadratic residues. In *IMACC*, volume 2260 of *Lecture Notes in Computer Science*, pages 360–363. Springer, 2001.
24. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *SP*, pages 910–927. IEEE, 2020.

25. Yevgeniy Dodis, Leonid Reyzin, and Adam D. Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In *EUROCRYPT*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540. Springer, 2004.
26. Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. Mcfly: Verifiable encryption to the future made practical. In *FC (1)*, volume 13950 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2023.
27. Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In *EUROCRYPT (2)*, volume 14652 of *Lecture Notes in Computer Science*, pages 311–341. Springer, 2024.
28. Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981, 2024.
29. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC*, pages 197–206. ACM, 2008.
30. Rishab Goyal and Vinod Vaikuntanathan. Locally verifiable signature and key aggregation. In *CRYPTO (2)*, volume 13508 of *Lecture Notes in Computer Science*, pages 761–791. Springer, 2022.
31. Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *CCS*, pages 89–98. ACM, 2006.
32. George Lu and Brent Waters. How to sample a discrete gaussian (and more) from a random oracle. In *TCC (2)*, volume 13748 of *Lecture Notes in Computer Science*, pages 653–682. Springer, 2022.
33. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, 2012.
34. Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. Fairblock: Preventing blockchain front-running with minimal overheads. In *SecureComm*, volume 462 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pages 250–271. Springer, 2022.
35. Chris Peikert. Bonsai trees (or, arboriculture in lattice-based cryptography). *IACR Cryptol. ePrint Arch.*, page 359, 2009.
36. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, pages 84–93. ACM, 2005.
37. Michael Reiter and Ken Birman. How to securely replicate services. *ACM TOPLAS*, 16(3):986–1009, 1994.
38. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *EUROCRYPT*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473. Springer, 2005.
39. Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.
40. Adi Shamir. Identity-based cryptosystems and signature schemes. In *CRYPTO*, volume 196 of *Lecture Notes in Computer Science*, pages 47–53. Springer, 1984.
41. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997.
42. Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. *IACR Cryptol. ePrint Arch.*, page 762, 2024.
43. Hong Wang, Yuqing Zhang, and Dengguo Feng. Short threshold signature schemes without random oracles. In *INDOCRYPT 2005*, volume 3797 of *Lecture Notes in Computer Science*, pages 297–310. Springer, 2005.
44. Brent Waters. Efficient identity-based encryption without random oracles. In *EUROCRYPT*, volume 3494 of *Lecture Notes in Computer Science*, pages 114–127. Springer, 2005.
45. Brent Waters, Hoeteck Wee, and David J. Wu. New techniques for preimage sampling: Improved nizks and more from LWE. *IACR Cryptol. ePrint Arch.*, page 1401, 2024. Eurocrypt 2025.
46. Hoeteck Wee. Circuit ABE with poly(depth, λ)-sized ciphertexts and keys from lattices. *IACR Cryptol. ePrint Arch.*, page 1416, 2024. CRYPTO 2024.
47. Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In *EUROCRYPT (3)*, volume 14006 of *Lecture Notes in Computer Science*, pages 385–416. Springer, 2023.
48. Mark Zhandry. To label, or not to label (in generic groups). In *CRYPTO (3)*, volume 13509 of *Lecture Notes in Computer Science*, pages 66–96. Springer, 2022.
49. Richard Zippel. Probabilistic algorithms for sparse polynomials. In *EUROSAM*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.

A Extended Preliminaries

A.1 Batch Threshold Decryption

Definition 13. A batch threshold decryption (BTD) scheme instantiated with a committee of $N \in \mathbb{N}$ parties, a threshold $t \in [N]$, a maximum batch size of $\ell \in \mathbb{N}$, a message space $\mathcal{M}$ and an identity space $\mathcal{I}$ consists of four PPT algorithms:

- $\text{BTD.SETUP}(1^\lambda, 1^\ell, 1^N, 1^t) \rightarrow (\text{pk}, \text{sk}_1, \dots, \text{sk}_N)$: The setup algorithm takes as input the security parameter λ , the maximum batch size ℓ , the number of parties N and the threshold t , and outputs a public key pk and N secret keys $(\text{sk}_1, \dots, \text{sk}_N)$.
- $\text{BTD.ENC}(\text{pk}, r, \mathbf{m}) \rightarrow \text{ct}$: The encryption algorithm takes as input a public key pk , an identity $r \in \mathcal{I}$ and a message $\mathbf{m} \in \mathcal{M}$, and outputs a ciphertext ct .
- $\text{BTD.PREDEC}(\text{sk}_i, (r_i)_{i \in [\ell]}) \rightarrow \text{sbk}_i$: The pre-decryption algorithm takes as input a secret key sk_i for a party $i \in [N]$ and ℓ identities, and outputs a pre-decryption key sbk_i for the party i .
- $\text{BTD.COMBINE}(\text{pk}, T \subseteq [N], (r_i)_{i \in T}, (\text{sbk}_i)_{i \in T}) \rightarrow \text{sbk}$: The combiner algorithm takes as input a subset $T \subseteq [N]$, ℓ identities, and the pre-decryption keys $(\text{sbk}_i)_{i \in T}$ of $|T| = t$ parties, and outputs a single short pre-decryption key sbk .
- $\text{BTD.DEC}(\text{pk}, \text{sbk}, (\text{ct}_i)_{i \in [\ell]}) \rightarrow (\mathbf{m}_i)_{i \in [\ell]}$: The decrypt algorithm takes as input a combined pre-decryption key sbk and ℓ ciphertexts, and outputs the corresponding messages.

Definition 14 (Correctness of BTD). For all $\lambda, \ell \in \mathbb{N}$, $T \subseteq [N]$, $(\mathbf{m}_i)_{i \in [\ell]} \in \mathcal{M}^\ell$ and $(r_i)_{i \in [\ell]} \in \mathcal{I}^\ell$, the following probability is greater than $1 - \text{negl}(\lambda)$:

$$\Pr \left[\text{BTD.DEC}(\text{pk}, \text{sbk}, (\text{ct}_i)_{i \in [\ell]}) = (\mathbf{m}_i)_{i \in [\ell]} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{SETUP}(1^\lambda, 1^\ell, 1^N, 1^t), \\ \text{ct}_i \leftarrow \text{BTD.ENC}(\text{pk}, r_i, \mathbf{m}_i), \\ \text{sbk}_i \leftarrow \text{BTD.PREDEC}(\text{sk}_i, (r_i)_{i \in [\ell]}) \quad \forall i \in [\ell] \\ \text{sbk} \leftarrow \text{BTD.COMBINE}(\text{pk}, T, (r_i)_{i \in T}, (\text{sbk}_i)_{i \in T}) \end{array} \right]$$

Definition 15 (Security of BTD). The security game $\text{Game}_{\mathcal{A}, b}^{\text{BTD}}(1^\lambda, \ell, t, N)$ is defined in [Figure 4](#) with respect to the adversary $\mathcal{A}$. A BTD scheme is selectively secure in the random oracle model, if for all $\ell, N = \text{poly}(\lambda)$, $t < N$ and PPT adversaries $\mathcal{A}$, there exists some negligible function $\text{negl}(\cdot)$ such that

$$|\Pr[\text{Game}_{\mathcal{A}, 0}^{\text{BTD}}(1^\lambda, \ell, t, N) = 1] - \Pr[\text{Game}_{\mathcal{A}, 1}^{\text{BTD}}(1^\lambda, \ell, t, N) = 1]| < \text{negl}(\lambda)$$

A.2 BEAT-MEV: Batch Threshold Decryption by Bormet, Faust, Othman, Qu

Bormet, Faust, Othman and Qu introduced the first efficient batch threshold decryption scheme without epochs [18]. At the core of BEAT-MEV is a *key-homomorphic puncturable PRF*, adapted from the PRF by Dujmovic et al. [27]. Each encryptor evaluates the PRF at a unique identity i using an ephemeral PRF key k_i , and masks its message by the PRF output: $\gamma_i = \mathbf{m}_i + \text{PRF}(k_i, i)$. The ciphertext has three elements: the masked message γ_i , an ElGamal encryption of the key k_i (in the exponent), and a punctured key k_i^* that is punctured at the identity i . The pre-decryption key sbk , created for a batch of ℓ identities $\{1, \dots, \ell\}$, is computed using the homomorphic property of ElGamal encryption and satisfies $\text{PRF}(\text{sbk}, i) = \sum_{j \in [\ell]} \text{PRF}(k_j, i)$. Finally, decryption recovers $\mathbf{m}_i$ by calculating

$$\gamma_i - \text{PRF}(\text{sbk}, i) + \sum_{j \in [\ell], j \neq i} \text{PuncturedPRF}(k_j^*, i),$$

The Security Game $\text{Game}_{\mathcal{A},b}^{\text{BTD}}(1^\lambda, \ell, t, N)$

Challenge identity: The adversary commits to the challenge identity r^* and sends it to the challenger.

Setup: The adversary sends polynomially bounded parameters $N, t \leq N$ and a subset $S \subseteq [N]$ of size $t - 1$ to the challenger. The challenger also takes as input the security parameter λ and the batch size ℓ , and runs $(\text{pk}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{BTD.SETUP}(1^\lambda, 1^\ell, 1^N, 1^t)$. It sends pk and $(\text{sk}_i)_{i \in S}$ to $\mathcal{A}$.

Pre-challenge queries: The adversary may issue an arbitrary number of pre-challenge queries for computing the pre-decryption key:

- $\mathcal{A}$ sends a list $(r_i)_{i \in [\ell]}$ of up to ℓ identities to Ch.
- Ch computes all N pre-decryption key shares for $i \in [N]$, $\text{sbk}_i \leftarrow \text{BD.PREDEC}(\text{sk}_i, (r_i)_{i \in [\ell]})$, and sends $(\text{sbk}_i)_{i \in [N]}$ to $\mathcal{A}$.

Challenge round: At any point in time, the adversary may decide that the current round is the challenge round. In this case,

- $\mathcal{A}$ sends two messages $\text{m}_0, \text{m}_1 \in \mathcal{M}$ on which, it would like to be challenged.
- If the adversary has previously asked for the pre-decryption key shares corresponding to a list of identities $(r_i)_{i \in [\ell]}$ such that $r^* \in (r_i)_{i \in [\ell]}$, then Ch halts the game.
- Otherwise, Ch computes $\text{ct}_b \leftarrow \text{BD.ENC}(\text{pk}, r^*, \text{m}_b)$ and sends ct_b to $\mathcal{A}$.

Post-challenge queries: The adversary may again issue an arbitrary number of key computation queries as in the pre-challenge case. If it asks for the pre-decryption key shares corresponding to a list of identities $(r_i)_{i \in [\ell]}$ such that $r^* \in (r_i)_{i \in [\ell]}$, then Ch aborts the game.

Random oracle queries: Upon receiving a query containing a new input x , Ch samples a value y from the output distribution of the random oracle, stores the tuple (x, y) , and returns y . It returns the value y upon receiving the same input x in a future query.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0, 1\}$. The game then halts with the same output b' .

Fig. 4: The security game for batch threshold decryption.

since $\text{PRF}(\text{sbk}, i) = \sum_{j \in [\ell]} \text{PRF}(k_j, i)$, and $\text{PuncturedPRF}(k_j^*, i) = \text{PRF}(k_j, i)$ for $j \neq i$.

The pre-decryption key is a single BLS12-381 $\mathbb{G}_2$ group element, and each ciphertext contains 3 group elements from $\mathbb{G}_1$ (i.e., the punctured key and the ElGamal ciphertext) and one group element from $\mathbb{G}_T$ (i.e., γ). However, the public keys for the key-homomorphic puncturable PRF have size quadratic in the domain of the PRF. Although the domain can be restricted to ℓ integers, as the success of decryption requires the PRF keys to be punctured at unique integers, this would require advance coordination among the encryptors. To remove coordination, the authors propose splitting a batch into multiple sub-batches, each containing ciphertexts with distinct identities, and publishing pre-decryption keys for each sub-batch (cf., [18, §5.3] for numerical analysis, this technique is called SB in Table 1). Since the decryption algorithm requires $O(\ell^2)$ pairings in total, this technique also reduces the computational burden of decryption, however at the expense of larger pre-decryption keys and ciphertexts.

Remark 8 (Succinctness Vulnerability for the Pre-decryption Key Size in SB). The sub-batches (SB) technique described above introduces a succinctness vulnerability for the pre-decryption key size. An adversary encryptor can submit ℓ ciphertexts, each created using the same integer as the PRF input. Then, the block proposer must publish $O(\ell)$ pre-decryption keys to enable the decryption of this batch of ciphertexts. This attack cannot be prevented by forcing the encryptor to choose the PRF inputs at random via a random oracle; since it can simply query the oracle at $O(\ell^2)$ points to find ℓ pre-images with the same output.

Lattice-based BEAT-MEV. We briefly describe the version of BEAT-MEV that uses the puncturable key-homomorphic PRF construction of Boneh, Lewi, Montgomery, and Raghunathan (BLMR) [16, §5] and estimate its parameter sizes.

The public parameters of the BLMR PRF consist only of two random matrices $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{m \times m}$, and its key is a vector $\text{sk} \in \mathbb{Z}_q^m$. The PRF is evaluated at a point $x = x_1 \dots, x_\ell \in \{0, 1\}^k$ as follows:

$$\text{PRF}(\text{sk}, x) = \left\lfloor \prod_{i=1}^k \mathbf{A}_{x_i} \cdot \text{sk} \right\rfloor_p,$$

This ensures that $\text{PRF}(\text{sk}_1 + \text{sk}_2, x) = \text{PRF}(\text{sk}_1, x) + \text{PRF}(\text{sk}_2, x) + e$, where the error term is $e \in \{0, 1, 2\}^m$. Here, the security parameter is n , and $q = O(\sqrt{n}2^{n^\epsilon})$, where $\epsilon \in [0, 1]$ links the LWE problem to the problem of approximating the worst-case GapSVP to $\tilde{O}(n/2^{n^\epsilon})$ factors. Moreover, $m = \lceil n \log q \rceil$, $k = n^\epsilon / \log(n)$ and $p = 2^{n^\epsilon - \omega(\log(n))}$, which implies $m = O(kn \log(n))$ and $\log(p) = O(k \log(n))$.

One can view the evaluation of the BLMR PRF as calculating a single branch in a tree of evaluations, whose leaf at an index x is the PRF evaluated at index x . Let $u_{\perp, x_1, \dots, x_{k'}}$, $k' \leq k$, denote the inner node at depth k' of the tree that is reached from the root by moving to the left child at depth i , if $x_i = 0$, and to the right child, if $x_i = 1$, proceeding through the depths $i = 1, \dots, k$. Then,

$$u_{\perp, x_1, \dots, x_{k'}} := \left\lfloor \prod_{i=1}^{k'} \mathbf{A}_{x_i} \cdot \text{sk} \right\rfloor_p,$$

Now, the BLMR PRF can be punctured at a point x by publishing as the punctured key, all the inner nodes that correspond to the siblings of the nodes on the path from the leaf x to the root. In other words, the punctured key consists of $u_{\perp, \overline{x_1}}, u_{\perp, x_1, \overline{x_2}}, \dots, u_{\perp, x_1, \dots, x_{k-1}, \overline{x_k}}$. This enables

calculating the PRF at any value $x' \neq x$ that first differs from x at some index j (i.e., $x'_i = x_i$ for $i < j$, and $x'_j \neq x_j$) as follows:

$$\text{PRF}(\text{sk}, x) = \left[u_{\perp, x_1, \dots, x_{j-1}, \bar{x}_j} \prod_{i=j+1}^k \mathbf{A}_{x_i} \right]_{p'},$$

where $p' = O(m^k p)$ is made larger to account for the increase in the noise when multiplying $\prod_{i=j+1}^k \mathbf{A}_{x_i}$ with the error term e associated with the inner node. This implies $\log(p') = k \log(m) + \log(p) = O(k(\log(k) + \log(n)))$.

Since BEAT-MEV requires the encryptors to publish their encrypted PRF keys through a homomorphic encryption scheme, we must also pick an appropriate homomorphic scheme for encrypting the BLMR PRF keys. We choose Regev encryption to preserve the structure of the PRF keys [36]. This also enables thresholdizing the new BEAT-MEV scheme by using a threshold version of Regev encryption [5].

Finally, we express the parameter sizes of the batch decryption scheme in terms of our notation for the security parameter, namely λ . As the matrices $\mathbf{A}_0$ and $\mathbf{A}_1$ are random, they can be generated from a small seed of size $O_\lambda(1)$ using a pseudo-random generator. Since Regev encryption has public keys of size $O_\lambda(1)$, the public key of the batch decryption scheme becomes $O_\lambda(1)$. However, the ciphertext now contains k vectors in $\mathbb{Z}_{p'}^m$:

$$\begin{aligned} |\text{ct}| &= km \log(p') = O(k^3 n \log(n)(\log(k) + \log(n))) \\ &= O(k^3 \lambda (\log^2(\lambda) + \log(k) \log(\lambda))) = O_\lambda(k^3). \end{aligned}$$

Finally, the pre-decryption key is simply the sum of the k PRF keys:

$$|\text{sbk}| = m \log q = O(k^2 n \log^2(n)) = O(k^2 \lambda \log^2(\lambda)) = O_\lambda(k^2).$$

B Explainable SampleLeft and the Proof of Theorem 1

We first present the definition of the explainable SamplePre sampler from [19, Definition 4.1]:

Definition 16 (Explainable SamplePre Sampler, Definition 4.1 of [19]). *Let n, m, q be lattice parameters. A $(\rho, \sigma_{\text{loss}})$ -explainable SamplePre sampler Π_{SP} with randomness length $\rho(\lambda, n, m, q)$ and Gaussian width loss $\sigma_{\text{loss}}(\lambda, n, m, q)$ is a pair of efficient algorithms (SamplePre, ExplainSP) with the following syntax:*

- **SamplePre**($\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{u}, \sigma; \text{rb}$) $\rightarrow \mathbf{v}$: *On input a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a gadget trapdoor $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m'}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a Gaussian width parameter σ , and randomness $\text{rb} \in \{0, 1\}^{\rho(\lambda, n, m, q)}$, the sampling algorithm deterministically outputs a vector $\mathbf{v} \in \mathbb{Z}_q^m$.*
- **ExplainSP**($1^\kappa, \mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{u}, \mathbf{v}, \sigma$) $\rightarrow \text{rb}$: *On input a precision parameter κ , a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a gadget trapdoor $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m'}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a preimage $\mathbf{v} \in \mathbb{Z}_q^m$, and a Gaussian width parameter σ , the explain algorithm outputs a bit string $\text{rb} \in \{0, 1\}^{\rho(\lambda, n, m, q)}$.*

We require the following two properties to hold except with negligible probability for all $\lambda \in \mathbb{N}$, all matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m'}$ such that $\mathbf{A}\mathbf{T}_{\mathbf{A}} = \mathbf{G}$, all target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, where $\|\mathbf{u}\|_\infty \leq 2^\lambda$, and all Gaussian width parameters σ , where $\|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\text{loss}}(\lambda, n, m, q) \leq \sigma \leq 2^\lambda$:

- **Correctness:** The following distributions are statistically close:

$$\{\mathbf{v} := \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb}); \text{rb} \leftarrow \{0, 1\}^\rho\} \quad \text{and} \quad \{\mathbf{v} \leftarrow \mathbf{F}_\sigma^{-1}(\mathbf{v})\},$$

Moreover, for all $\mathbf{v}$ in the support of $\text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb})$, we have $\mathbf{A}\mathbf{v} = \mathbf{u}$.

- **Explainability:** For all precision parameters $\kappa \in \mathbb{N}$, the statistical distance between the following distributions is bounded by $1/\kappa + \text{negl}(\lambda)$:
 - **DSamplePre:** Sample $\text{rb} \leftarrow \{0, 1\}^\rho$ and set $\mathbf{v} := \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb})$. Output $(\mathbf{v}, \text{rb})$.
 - **DExplain:** Sample $\text{rb}' \leftarrow \{0, 1\}^\rho$ and set $\mathbf{v} := \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb}')$. Find

$$\text{rb} := \text{ExplainSP}(1^\kappa, \mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma)$$

and output $(\mathbf{v}, \text{rb})$.

Theorem 6 (Theorem 4.2 of [19]). There exists a $(\rho, \sigma_{\text{loss}})$ -explainable **SamplePre** sampler for

$$\rho = \text{poly}(\lambda, n, m, \log q)$$

and

$$\sigma_{\text{loss}}(\lambda, n, m, q) = 18m^{3/2} \log(m\lambda) \log(\log(q)).$$

Let $\Pi_{\text{SP}} = (\text{SamplePre}, \text{ExplainSP})$ denote the $(\rho, \sigma_{\text{loss}})$ -explainable **SamplePre** sampler at [19, Construction 7.6], which satisfies **Theorem 6**.

Our explainable **SampleLeft** sampler construction also relies on the following lemma:

Lemma 25 (Lemma 7.4 of [19], From [33]). Let n, q be lattice parameters and let $m' = n \lceil \log(q) \rceil$. Take any $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m'}$ such that $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$. Let

$$\mathbf{T}'_\mathbf{A} = [\mathbf{I}_m - \mathbf{T}_\mathbf{A}\mathbf{G}^{-1}\mathbf{A} \mid \mathbf{T}_\mathbf{A}\mathbf{S}] \in \mathbb{Z}_q^{m \times (m+m')},$$

where $\mathbf{S} \in \mathbb{Z}_q^{m' \times m'}$ is an Ajtai trapdoor for $\mathbf{G}$ (full-rank basis for $\Lambda^\perp(\mathbf{G})$). Then $\mathbf{A}\mathbf{T}'_\mathbf{A} = \mathbf{0} \pmod{q}$ and $\mathbf{T}'_\mathbf{A}$ is full rank over $\mathbb{R}$.

B.1 The Explainable **SampleLeft** Construction

SampleLeft $(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \text{rb})$. Let $\mathbf{T}'_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$ denote the first m linearly-independent (in $\mathbb{R}$) columns of $[\mathbf{I}_m - \mathbf{T}_\mathbf{A}\mathbf{G}^{-1}\mathbf{A} \mid \mathbf{T}_\mathbf{A}\mathbf{S}]$, where $\mathbf{S} \in \mathbb{Z}_q^{m \times m}$ is the Ajtai trapdoor for $\mathbf{G}$ (full-rank basis for $\Lambda^\perp(\mathbf{G})$) from **Lemma 7**. Note that $\mathbf{T}'_\mathbf{A}$ is an Ajtai trapdoor for $\mathbf{A}$. The sampling algorithm first finds this trapdoor $\mathbf{T}'_\mathbf{A}$. It then uses the basis extension algorithm **ExtBasis** [35] to construct an Ajtai trapdoor $\mathbf{T}'_\mathbf{F} \in \mathbb{Z}^{(m+m_1) \times (m+m_1)}$ for the vector $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$ such that $\|\mathbf{T}'_\mathbf{F}\|_{\text{GS}} = \|\mathbf{T}'_\mathbf{A}\|_{\text{GS}}$ ([35, Lemma 3.2]). Crucially, it does this in a *deterministic* manner by calling $\mathbf{T}'_\mathbf{F} = \text{DetExtBasis}(\mathbf{T}'_\mathbf{A}, [\mathbf{A} \mid \mathbf{M}_1])$ described below:

- For $i = 1, \dots, m$, it sets $\mathbf{s}'_i := \mathbf{S}[i] \parallel \mathbf{0} \in \mathbb{Z}^{m+m_1}$, where $\mathbf{S}[i] \in \mathbb{Z}^m$ denotes the vector at the i -th column of the matrix $\mathbf{S}$.
- For $i = 1, \dots, m_1$, it uses Gaussian elimination to compute a vector $\mathbf{t}_i \in \mathbb{Z}^m$ such that $\mathbf{A}\mathbf{t}_i = -\mathbf{M}_1[i] \in \mathbb{Z}_q^n$, where $\mathbf{M}_1[i]$ denotes the vector at the i -th column of the matrix $\mathbf{M}_1$. It then sets $\mathbf{s}'_{m+i} := \mathbf{t}_i \parallel \mathbf{e}_i \in \mathbb{Z}^{m+m_1}$, where $\mathbf{e}_i \in \mathbb{Z}^{m_1}$ is the i -th standard basis vector.

Finally, the sampling algorithm outputs the vector returned by **SamplePre** $(\mathbf{F}, \mathbf{T}'_\mathbf{F}, \mathbf{u}, \sigma; \text{rb})$ from Π_{SP} , where $\mathbf{T}_{\text{Ajtai}}$ at [19, Construction 7.6] is replaced by $\mathbf{T}'_\mathbf{F}$.

$\text{ExplainSL}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma)$. The algorithm first obtains $\mathbf{T}_\mathbf{F} = \text{DetExtBasis}(\mathbf{T}_\mathbf{A}, [\mathbf{A} \mid \mathbf{M}_1])$ *deterministically* as described above. It then returns the output of

$$\text{ExplainSP}(1^\kappa, \mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \mathbf{v}, \sigma)$$

from Π_{SP} , where $\mathbf{T}_{\text{Ajtai}}$ at [19, Construction 7.6] is replaced by $\mathbf{T}'_\mathbf{F}$.

B.2 Proof of Theorem 1

Finally, we prove that Π_{SL} described above is an explainable SampleLeft sampler.

Proof of Theorem 1. By Lemma 25, $\mathbf{T}'_\mathbf{A}$ is a full-rank Ajtai trapdoor for $\mathbf{A}$, and by [35, Lemma 3.2], $\mathbf{T}'_\mathbf{F}$ is an Ajtai trapdoor for $\mathbf{F}$ such that $\|\mathbf{T}'_\mathbf{F}\|_{\text{GS}} = \|\mathbf{T}'_\mathbf{A}\|_{\text{GS}}$. Moreover, it is full-rank.

Proof of correctness of Π_{SP} requires $\|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\text{loss}}$ to be lower-bounded by the ℓ_2 -norm of the Gram-Schmidt orthogonalization of $\mathbf{T}'_\mathbf{F}$ (i.e., $\|\mathbf{T}'_\mathbf{F}\|_{\text{GS}}$) as follows:

$$\|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\text{loss}} \geq \|\mathbf{T}'_\mathbf{F}\|_{\text{GS}} \cdot (\log((m + m_1)\lambda) + 2 \log(m + m_1) + \log(\log(q)) + 5).$$

Since we know that $\|\mathbf{T}'_\mathbf{F}\|_{\text{GS}} = \|\mathbf{T}'_\mathbf{A}\|_{\text{GS}}$, we can replace $\|\mathbf{T}'_\mathbf{F}\|_{\text{GS}}$ by $\|\mathbf{T}'_\mathbf{A}\|_{\text{GS}}$ in the requirement:

$$\|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\text{loss}} \geq \|\mathbf{T}'_\mathbf{A}\|_{\text{GS}} \cdot (\log((m + m_1)\lambda) + 2 \log(m + m_1) + \log(\log(q)) + 5)$$

Now, by the proof of Theorem 6, this requirement is satisfied by $\sigma_{\text{loss}}(\lambda, n, m, m_1, q) = 18(m + m_1)^{3/2} \log((m + m_1)\lambda) \log(\log(q))$. Therefore, by Theorem 6, given this value of $\sigma_{\text{loss}}(\lambda, n, m, m_1, q)$ and $\rho = \text{poly}(\lambda, n, m + m_1 \log(q)) = \text{poly}(\lambda, n, m, m_1 \log(q))$, Π_{SP} satisfies correctness and explainability per Definition 16:

1. The statistical distance between the distributions $\mathbf{v} := \text{SamplePre}(\mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \sigma; \text{rb})$, $\text{rb} \leftarrow \{0, 1\}^\rho$ and $\mathbf{v} \leftarrow \mathbf{F}_\sigma^{-1}(\mathbf{u})$ is negligible in λ , and for all $\mathbf{v}$ in the support of $\text{SamplePre}(\mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \sigma; \text{rb})$, it holds that $\mathbf{F}\mathbf{v} = \mathbf{u}$.
2. For all $\kappa \in \mathbb{N}$, the statistical distance between the following distributions is bounded by $1/\kappa + \text{negl}(\lambda)$:
 - DSamplePre: Sample $\text{rb} \leftarrow \{0, 1\}^\rho$ and $\mathbf{v} := \text{SamplePre}(1^\lambda, \mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \sigma; \text{rb})$. Output $(\mathbf{v}, \text{rb})$.
 - DExplainPre: Sample $r' \leftarrow \{0, 1\}^\rho$ and $\mathbf{v} := \text{SamplePre}(1^\lambda, \mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \sigma; \text{rb}')$. Compute

$$\text{rb} := \text{ExplainSP}(1^\lambda, 1^\kappa, \mathbf{F}, \mathbf{T}_\mathbf{F}, \mathbf{u}, \mathbf{v}, \sigma)$$

and output $(\mathbf{v}, \text{rb})$.

Finally, since the algorithm DetExtBasis is deterministic, correctness and explainability of Π_{SL} per Definition 6 directly follow from items (1) and (2). $\square$